

Manipulation in the Dark: Blind Sim-to-Real Transfer of a Baoding Policy on the Shadow Hand Lite

Ayush Deshmukh



Master of Science
School of Informatics
University of Edinburgh
2026

Abstract

Dexterous in-hand manipulation is a demanding robotics problem: the object is constantly moving, contacts shift across the hand, and the dynamics change throughout the motion. The Baoding ball rotation task captures this difficulty, requiring two free-floating objects to be continuously rotated through ball-to-ball and ball-to-hand contact with no terminal success state. Reinforcement Learning in simulation provides a scalable way to train policies for such tasks without the cost and risk of training directly on hardware. Previous work has shown that tactile sensing can improve dexterous manipulation in simulation, but these results have largely remained in simulation. Transferring such policies to real systems introduces a sim-to-real gap arising from differences in sensing, actuation, and contact dynamics. In this work, we study this gap for a blind Baoding policy on the Shadow Dexterous Hand Lite, using only proprioception and binary tactile observations, with no vision or hardware training data. We decompose the gap into proprioception, tactile, and dynamics components and systematically narrow these gaps to improve hardware transfer. The resulting policy achieves 55 full 180° rotations in a single continuous episode without dropping either ball, reaching up to six rotations per 10 seconds. To our knowledge, this is the strongest reported Baoding result on hardware in both rotation rate and sustained operation, despite the policy being blind.

Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(Ayush Deshmukh)

Acknowledgements

This work was carried out under the supervision of Elle Miller and Professor Sethu Vijayakumar at the Statistical Learning and Motor Control Group, University of Edinburgh. I thank Nalin Agarwal for help building the FSR hardware setup and calibration pipeline, and Shadow Robotics for helping identify the tendon-slack behaviour on the hand.

Table of Contents

1	Introduction	1
1.1	Motivation	1
1.2	Contributions	2
2	Background and Related Work	3
2.1	Two Routes to a Dexterous Policy	3
2.2	Blind Manipulation, Tactile Sensing, and RoTO	5
2.3	The Sim-to-Real Gap and the Position of This Work	6
3	Problem Statement	8
4	Hardware and Simulation Setup	10
4.1	Hardware Setup	10
4.2	Simulation Setup	11
4.2.1	Coupling Mechanism	12
5	Quantifying the Sim-to-Real Gap	14
5.1	Proprioception Gap	14
5.2	Tactile Gap	16
5.3	Dynamics Gap	17
6	Narrowing the Sim-to-Real Gap	18
6.1	Narrowing the Proprioception Gap	18
6.1.1	Actuator Identification	18
6.1.2	Coupled-Joint Investigation	19
6.1.3	Learned Actuator Dynamics	21
6.1.4	Open-loop Replay and Task Scoping	24
6.2	Narrowing the Tactile Gap	26

6.2.1	Addressing Calibration and Offset	26
6.2.2	Tactile Corruption	26
6.3	Narrowing the Dynamics Gap	27
6.3.1	Closed-loop Ablation Harness	27
6.3.2	Dynamics Randomisation	28
7	Results and Analysis	29
7.1	Actuator Identification	29
7.2	Learned Actuator Model	30
7.2.1	Unsupervised Actuator Networks	30
7.2.2	Physics-based Position Loss	30
7.2.3	Learning Torque from Raw Uncalibrated Signals	32
7.2.4	Open-loop Replay	33
7.3	Hardware Transfer Progression	34
7.3.1	Proprioception-Only policy	35
7.3.2	Proprioception and Tactile Policy	37
8	Conclusion	40
	Bibliography	41
A	Appendices	43
A.1	Supplementary videos	43
A.2	Joint-limit table for the Shadow Hand Lite	44
A.3	Full identified actuator gains	44
A.4	Isaac Sim rollout-stall diagnosis	45
A.5	Excitation-signal specification and Latin Hypercube configuration	46
A.6	Data pipeline in full	47
A.7	UAN full specification	48
A.8	Generalized Actuator Nets (Position Loss approach) full specification	50
A.9	Encoder fine-tuning attempt	51
A.10	Configurations, checkpoints and DR profiles	53

Chapter 1

Introduction

Humans manipulate objects within their hands with remarkable ease, reorienting, rolling, and repositioning them through touch alone. Replicating this in robotic systems remains one of the hardest open problems in robotics. What makes dexterous in-hand manipulation harder than a simple pick-and-place task is that the object is moved relative to the hand rather than relative to the world. The hand never controls the object directly; it must continuously coordinate several fingers while establishing, breaking, and redistributing contact, with the dynamics changing every time a finger makes or breaks contact.

Baoding ball rotation is a good example of this problem. The task is a traditional Chinese practice in which two balls are rotated around each other within a single hand without dropping either one. It is harder than single-object reorientation: two free-floating objects must be manipulated at once, contact occurs between the two balls as well as between the balls and the hand, and there is no terminal success state. Small modelling errors are therefore never washed out by reaching a goal, and the policy must keep the balls turning under continuous contact throughout the motion. Performance is graded by the number of rotations rather than as a binary success, which lets us measure the size of the transfer gap across methods.

1.1 Motivation

Reinforcement learning (RL) in simulation has become the standard way to train policies for tasks like this, offering parallelised training across thousands of environments without the cost and risk of real hardware. Training directly on the hand is slow, requires resetting after every dropped ball, and wears the mechanism. What simulation

gives in scale, however, it takes back in fidelity: actuator dynamics are idealised, contact is approximated under rigid-body assumptions, and the simulator provides perfect state where real sensors are noisy. This is the sim-to-real gap: a policy trained in simulation often fails on hardware because of discrepancies it never saw during training. Since we follow the simulation route, this gap is the central problem of this work.

We approach this gap in two ways: we identify where the simulated and real hand diverge, and we bring the simulation closer to hardware so that transfer becomes possible. To keep identification tractable we split the gap into three parts, each defined so it can be measured directly, and close it along the same three axes: the proprioception gap, the tactile gap, and the dynamics gap.

We take this a step further in a setting that makes the problem significantly harder: we deploy the policy *blind*, using proprioception and binary tactile observations only, with no camera, no depth, no motion capture, and no privileged simulator state at inference. In almost all prior work on Baoding the object state is supplied from outside the hand; removing that dependence makes transfer harder, not easier, since the policy has no external signal to correct against once the simulated and real hand begin to diverge.

Our aim is to transfer a blind, simulation-trained Baoding policy to the Shadow Dexterous Hand Lite without any hardware training data, and to characterise what must be corrected in simulation for that transfer to succeed. **Under this setting, our policy completed 55 full 180° rotations in a single episode without dropping either ball, and up to six rotations in ten seconds. To our knowledge this is the strongest blind Baoding result reported on physical hardware along both axes that matter for this task: rotation speed and sustained operation without a drop.**

1.2 Contributions

Towards blind sim-to-real transfer of a Baoding policy on the Shadow Dexterous Hand Lite, we (i) identify the actuator behaviour of all thirteen commandable joints from motion data (Section 6.1.1); (ii) correct the coupled-joint representation to match the tendon mechanism on hardware (Sections 4.2.1, 6.1.2); (iii) decompose the sim-to-real gap into proprioception, tactile, and dynamics components and narrow each through targeted randomisation of ball mass, friction, and command slew (Section 6.3) and tactile corruption (Section 6.2.2); and (iv) demonstrate a blind, simulation-trained policy that transfers to the Shadow Dexterous Hand Lite with no hardware training data.

Chapter 2

Background and Related Work

2.1 Two Routes to a Dexterous Policy

There are broadly two approaches to producing reliable dexterous behaviour on physical hardware. The first is to learn the behaviour directly on the hardware, no simulator and therefore no transfer step. Zhu et al. [16] used model-free reinforcement learning trained directly on the physical hand. They evaluated the approach on two hands: the DClaw, a custom three-fingered 9-DoF manipulator built from Dynamixel servos, and the Allegro hand, a four-fingered 16-DoF anthropomorphic hand. The tasks included valve turning, box flipping, and door opening, and each task required a lot of time to train. Box flipping required four hours, valve turning seven hours, and door opening 16 hours of training. Over 10 evaluation rollouts for each task, a 100% success rate was achieved.

This method was also extended to the Baoding ball rotation task. Nagabandi et al. [10] learned a dynamics model from real-world interaction and used online planning through model predictive control (MPC) to learn multiple manipulation tasks, including Baoding ball rotation on a 24-DoF Shadow Hand. Using only four hours of real-world data, they achieved a 100% success rate for a single 90° rotation and a 54% success rate for a full 180° rotation, with occasional multiple rotations also achieved. This work was among the first demonstrations that contact-rich manipulation could be learned directly on physical dexterous hardware. However, the system relied on an external stereo camera rig and a convolutional neural network tracker to estimate the 3D position of the balls.

The second method is to train a policy to mimic the intended behaviour in simulation and then transfer that behaviour to hardware. Andrychowicz et al. [2] used the 24-DoF

Shadow Dexterous Hand to learn in-hand block reorientation. They trained the policy entirely in simulation and transferred the behaviour directly to the hand without any real-world data. They used physical parameter randomisation (Domain Randomisation) in simulation to account for the dynamics gap and perception gap between simulation and hardware. They used three RGB cameras for object pose estimation plus motion capture markers. The method achieved 13 consecutive successful reorientations of the block on the physical hand.

This sim-to-real transfer is also possible for Baoding ball rotation. Yuan et al. [15] used vision and touch by projecting the tactile contact location into a 3D point cloud, similar to the 3D point cloud obtained from the depth camera. They used Isaac Gym to train the policy and an Allegro Hand to transfer the policy. Using Baoding ball rotation as the task, they showed that combining vision and touch performed better than either modality alone for both reinforcement learning performance and sim-to-real transfer. They reported a maximum of three complete rotations on hardware in a 10-second episode.

Methods that directly learn on hardware are slow to train, expensive to run, require repeatedly resetting the environment, and therefore incur high labour costs. They also risk wearing or damaging the hardware and do not scale easily across multiple tasks. Their advantage is that the policy is learned on the physical system itself, so nothing has to be assumed about the dynamics and there is no transfer step. Simulation allows training to scale and reduces the cost of learning directly on hardware. However, it comes at the cost of the sim-to-real gap: what happens in simulation is not what happens on hardware, and this difference has to be mitigated in some way.

In both cases the object state was supplied from outside the hand, whether the policy was learned directly on the robot [10] or transferred from simulation [15]. Results that remove this dependence and rely on proprioception alone [12] involve a single object, and therefore do not require the coordination between two moving objects that Baoding demands. This work follows the simulation route, since Baoding requires far more episodes and far more dropped balls than is practical to collect on a physical hand, and simulation is what makes that scale available. The cost of this choice is the sim-to-real gap, which this work therefore takes as its central problem, while also keeping the policy restricted to the sensing available on the hand itself.

2.2 Blind Manipulation, Tactile Sensing, and RoTO

The preceding section showed that the object state was in almost every case supplied from outside the hand. This is a practical constraint rather than a design constraint. Vision is a great source of information, but using cameras is not always practical for in-hand manipulation. The fingers can occlude the object, and this occurs most when the hand is enclosed around the object. Vision is also not the only modality humans use to manipulate objects in-hand, as we can manipulate objects blindly without necessarily needing vision. This raises the question of whether tactile sensing can provide the information needed to replace vision for these policies.

The choice of tactile representation itself introduces a transfer problem. There are two main choices here: high-dimensional tactile observations or simple binary tactile observations. High-dimensional observations provide richer information, but introduce additional challenges during transfer, including sensor construction, calibration, geometry replication, material properties, and placement in simulation. Binary contact sensing is significantly easier to simulate and transfer, but it only tells us whether an object is in contact with the sensor or not, and loses information about contact magnitude and distribution. Lee et al. [6] showed that binary contact sensing can reduce the sim-to-real gap while retaining enough information for manipulation. Binary sensing, however, is not universally preferable, as Pan et al. [11] showed that richer, physically grounded representations such as centre-of-pressure can preserve additional information useful for the task while still transferring zero-shot from simulation to hardware. Therefore, the choice is not simply between dense and sparse sensing, but depends on whether the task actually requires the additional information provided by dense tactile observations.

Miller et al. [7] combined 17 sparse binary tactile sensors with proprioception on the five-fingered, 24-DoF Shadow Dexterous Hand, and released the Robot Tactile Olympiad (RoTO), a benchmark for blind contact-rich manipulation in which reinforcement learning is used to learn the task without vision. Their agents achieved superhuman performance on Baoding ball rotation in simulation, reaching up to 25 rotations in a 10-second episode against a human expert’s 13. RoTO 2.0 [8] extended the benchmark to four morphologies, including the Shadow Dexterous Hand Lite used in this work. This established that binary contact sensing can provide sufficient information for Baoding ball over dense tactile representation. However, what this result does not address is whether the same policy transfers to physical hardware. The present work takes the RoTO formulation as its starting point and focuses on the sim-to-real

gap that appears when such a policy is deployed on the Shadow Dexterous Hand Lite.

2.3 The Sim-to-Real Gap and the Position of This Work

The sim-to-real gap in contact-rich dexterous manipulation has multiple sources, which can be broadly divided into three levels. The first is the robot level, where differences in actuator gains, damping, friction, tendon compliance, control rate, and mechanical coupling can cause the same command to produce a different trajectory in simulation and on hardware. The second is the observation level, where the same physical hand state can produce a different policy input due to calibration errors between simulation and hardware, sensor noise, temporal offsets, and sensor-specific failure modes. The third is the interaction level, where differences in contact friction, object mass, and inertia can change how the ball responds to the same hand motion. A policy trained only in simulation is exposed only to the simulated version of each of these factors: simulated actuator responses, simulated sensing, and simulated contact dynamics. As soon as it encounters the real-world hand, these differences can cause the policy to fail at deployment.

A major amount of prior work handles these differences implicitly through Domain Randomisation by training across a broad distribution of physical parameters. The policy therefore learns to perform the same task across a wider range of scenarios, with the hope that the real world scenario falls within this distribution represented during training [14, 2, 1]. This is effective because the policy does not need to be correct for one precise model. The main limitation is that randomisation does not identify which parameter is responsible for a given failure. For example, widening the range of object friction does not correct an incorrect actuator stiffness, while randomising a sensor value does not reproduce a mismatch caused by joint coupling.

In this work we therefore treat the sim-to-real gap as a problem of both identification and transfer. The transfer problem is separated into three major components, each defined so that it can be measured directly: the proprioception gap, defined by the difference in joint trajectories under the same command; the tactile gap, defined by the difference between binary contact observations in simulation and on hardware for a given trajectory; and the dynamics gap, defined by the difference in the physical parameters governing the ball. The objective is to measure the dominant mismatch and then make the smallest targeted changes needed to narrow it, all while preserving the blind agent formulation adopted from RoTO. Each gap is associated with a pri-

mary hypothesis, with the proprioception hypothesis being further refined as the work progresses through the dissertation. The hypotheses tested in this work are listed below.

Hypothesis 1 | Proprioception Gap

The proprioception gap can be reduced by identifying the physical actuator behaviour, i.e. K_p (Stiffness), K_d (Damping), and F_c (Coulomb friction), more accurately in simulation.

Hypothesis 1a | Coupled-Joint Behaviour

No combination of K_p , K_d , and F_c can reproduce the coupled-finger behaviour observed on hardware. The remaining mismatch arises from how the coupling mechanism is modelled in simulation compared with how the joints actually behave on hardware, and is thus a property of the coupling mechanism rather than of the identified actuator parameters.

Hypothesis 1b | Learned Actuator Dynamics

A learned residual model can represent the coupled-joint behaviour that the PD model cannot. Adding a learned correction to the simulated PD output should reproduce the early-unlock behaviour observed on hardware without requiring an explicit model of the tendon mechanism.

Hypothesis 1c | Open-loop Transfer

A simulation-trained trajectory will diverge from the physical hand when it enters the coupled-joint region ($100^\circ - 180^\circ$) because the remaining J1–J2 coupling mismatch is not represented in simulation.

Hypothesis 2 | Tactile Gap

The tactile gap can be reduced through accurate sensor calibration and by training the policy to be robust to the sensor noise and channel inconsistencies observed on hardware.

Hypothesis 3 | Dynamics Gap

The dynamics gap can be reduced by exposing the policy to variation in relevant physical parameters, such as ball mass and friction, during simulation training.

Chapter 3

Problem Statement

[Parts of this section have been taken verbatim from the IPP report.]

We formalise sim-to-real transfer as a mismatch between two Markov Decision Processes that share everything except their dynamics and their observation channels.

Domains. Let the simulation domain be an MDP

$$M_{\text{sim}} = (S, A, T_{\text{sim}}, R, \rho_0, \gamma),$$

and the real-world domain be

$$M_{\text{real}} = (S, A, T_{\text{real}}, R, \rho_0, \gamma),$$

where S is the state space, A the action space, $T : S \times A \rightarrow \Delta(S)$ the transition kernel ($\Delta(S)$ the probability distribution over S), $R : S \times A \rightarrow \mathbb{R}$ the reward function, ρ_0 the initial-state distribution, and $\gamma \in [0, 1)$ the discount factor. The two domains share S , A , R , ρ_0 , and γ ; they differ in the transition kernels T_{sim} and T_{real} , and in the observation channels described next.

Observations. In practice, the agent does not observe the full state $s \in S$. The observation functions

$$O_{\text{sim}} : S \rightarrow \Delta(Z), \quad O_{\text{real}} : S \rightarrow \Delta(Z),$$

map states to distributions over an observation space Z , and differ due to perceptual noise, sensor imperfections, and calibration errors, inducing a distribution shift over observations even for the same underlying state.

The domain gap. We define the sim-to-real gap as the joint mismatch in transition and observation functions,

$$\mathcal{G} = (T_{\text{sim}}, O_{\text{sim}}) \neq (T_{\text{real}}, O_{\text{real}}).$$

The transition gap $T_{\text{sim}}(s' | s, a) \neq T_{\text{real}}(s' | s, a)$ captures discrepancies in contact physics, actuator dynamics, and mechanical properties. The observation gap $O_{\text{sim}} \neq O_{\text{real}}$ captures sensor noise, calibration error, and perceptual distribution shift. Both are present in the deployment considered here: the physical hand does not evolve as the simulated hand does under the same action, and its sensors do not report what the simulator reports for the corresponding state.

Policy and objective. A stochastic policy $\pi : Z \rightarrow \Delta(A)$ is trained in simulation by maximising the expected discounted return,

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{M_{\text{sim}}} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right].$$

The sim-to-real transfer problem. The objective is to find π^* such that the performance loss upon deployment to the real world is minimised,

$$\min_{\pi} \underbrace{\mathbb{E}_{M_{\text{sim}}} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] - \mathbb{E}_{M_{\text{real}}} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]}_{\text{transfer gap } \delta(\pi)},$$

and transfer is considered successful when $\delta(\pi^*) \leq \epsilon$ for some acceptable tolerance $\epsilon \geq 0$.

Observation setting. We evaluate transfer under a blind observation setting: the policy receives proprioception and tactile observations only, with no camera, RGB image, or depth input at inference, and no privileged simulator state. The information available to the deployed policy is therefore restricted to what can be obtained from the physical hand and its tactile sensors.

Task. The task is continuous Baoding ball rotation with two identical smooth balls, each of diameter 1.58 in and mass 55 g, initialised diagonally opposed in the hand. Rotation is counted through ball–ball position exchanges: with the balls starting in the diagonally opposed configuration, one exchange is counted as 90° , and a second exchange, which returns the balls to the starting configuration, as 180° . An episode terminates when a ball is dropped, so the task requires the policy to sustain the manipulation continuously while keeping both balls within the hand. The task performance on hardware is measured along two axes: the number of rotations completed before the ball is dropped, and the total number of rotations completed within 10 seconds.

Chapter 4

Hardware and Simulation Setup

4.1 Hardware Setup

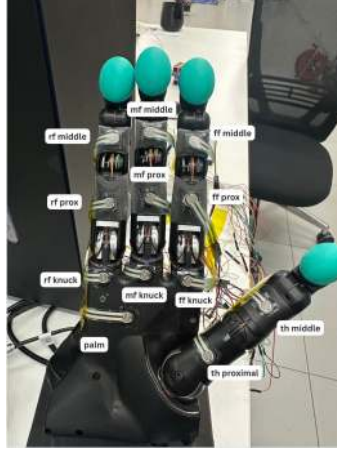
We use the Shadow Dexterous Hand Lite as the physical platform. The hand has 13 independently commandable actuators and 16 kinematic degrees of freedom. The three dependent joints FFJ1, MFJ1, and RFJ1 (see Figure 4.1b) are coupled to their corresponding J2 driver joints through the tendon-driven mechanism. The policy therefore produces 13 action values, while the underlying hand kinematics contain 16 joints. The joint-limit table is provided in Appendix A.2.

The physical hand used in this project is the Shadow Dexterous Hand Lite, fitted with BioTac fingertip sensors and twelve force-sensitive resistor (FSR) sensors distributed across fixed locations on the hand and forearm. Figure 4.1b shows the joint configuration and nomenclature used throughout, while Figure 4.1a shows the placement of the FSR sensors. We use this joint nomenclature consistently throughout the remainder of the dissertation.

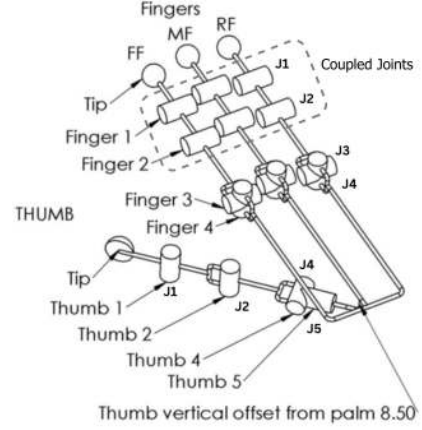
The BioTac sensors provide multimodal tactile measurements at the fingertips, while the FSR array provides contact measurements at locations away from the fingertips. The FSR calibration pipeline was developed separately and is used in this work as provided. The dense per-fingertip taxel representation is not used in the experiments reported here. The tactile input used by the policy is limited to the BioTac fingertip measurements and the twelve FSR channels. The hand configuration described above is not the one used for the results reported in the progress report, which relied on TouchLab 16-taxel fingertip arrays in place of the BioTac fingertips. The reasoning for the switch is given in Chapter 6 at the point in the narrative where it becomes relevant.

Closed-loop hardware deployment uses reduced proportional gains relative to the

default hand controller, chosen to keep tracking stable under the policy’s high-rate commands while leaving damping unchanged; open-loop replay uses the default gains. The full identified K_P/K_D values and per-joint group multipliers are given in Appendix A.3, Table A.2.



(a)



(b)

Figure 4.1: (a) Shadow Hand Lite equipped with BioTac fingertips (in turquoise) with the FSRs labelled. We use Twelve FSRs in this work. (b) Structural representation of the Shadow Hand Lite, showing the labeled joints for the ring finger and thumb. J1 and J2 joints on each finger are coupled.

4.2 Simulation Setup

The simulation is implemented using Isaac Sim [4] and Isaac Lab [9] within the Robot Tactile Olympiad (RoTO) environment. The physics simulation runs at 240 Hz with a control decimation of four, giving an effective control rate of 60 Hz. Each episode lasts up to ten seconds, corresponding to 600 control steps.

The reward contains a dense term based on the distance of each ball from its target and a sparse term awarded when both balls reach their targets within a 1 mm tolerance. Each successful target achievement increments the internal rotation counter and assigns a new pair of target positions. Episodes terminate when a ball leaves the reachable region or falls below the specified height, or when the ten-second timeout is reached. The policy consumes a stacked proprioceptive-and-tactile observation and outputs 13 joint-position targets that drive the Shadow Hand Lite; the resulting next state produces both the next observation and the reward that trains the policy with PPO (Fig. 4.2).

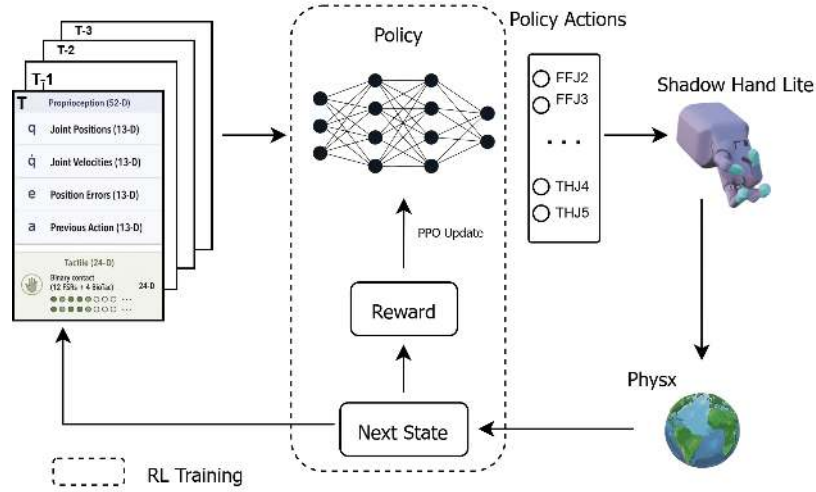


Figure 4.2: Blind control loop and RL training setup. At each control step the policy receives a stacked observation of the last four frames of proprioception (joint position, joint velocity, position error, and previous action) and binary tactile contact. It outputs 13 actions that are mapped to joint position targets and sent to the Shadow Hand Lite; the resulting physics (NVIDIA Physx) produces the next state, which forms the next observation. During training, the reward derived from the next state updates the policy with PPO (dashed box). No visual or privileged simulator state is used at any point.

The simulated hand is controlled using an implicit proportional-derivative actuator model. The per-joint effort limits are approximately 0.9–2.4 N·m. The training configuration uses $K_p = 1.0$ (Stiffness) and $K_d = 0.1$ (Damping) for all joints. These are placeholder values and are not the identified gains obtained from hardware data in Chapter 6. The two sets of experiments were maintained separately, and the identified gains were therefore not used in the training configuration of the deployed policy.

4.2.1 Coupling Mechanism

Our preliminary work, which was accepted to the International Conference on Robotics and Automation (ICRA) ViTac Workshop, assumed 16 independently actuated joints [8] (Video linked in Appendix A.1). Once the physical hand was verified to have 13 independently commandable actuators, the policy was retrained to produce 13 actions and coupling was handled during runtime.

In the default URDF/USD model used by Isaac Lab, J1 is represented as a continuous multiple of J2, causing the two joints to move together according to a fixed coupling ratio. This does not correspond to the physical mechanism of the Shadow Dexterous

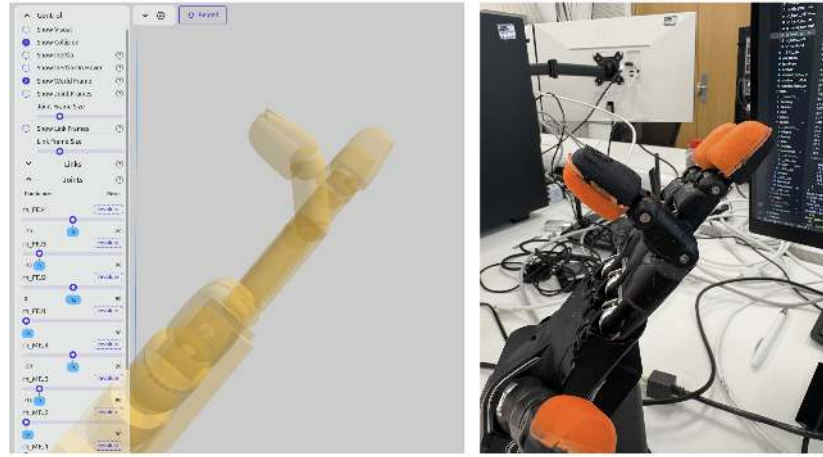


Figure 4.3: Coupled-joint behaviour at a commanded J2 of 45° . Left: default Isaac Sim URDF/USD, in which J1 tracks J2 in a fixed ratio and reaches 90° . Right: physical Shadow Hand Lite, in which J1 remains at 0° while J2 travels.

Hand Lite, in which a single tendon drives the pair sequentially: J2 travels first while J1 remains stationary, and J1 only begins to move once J2 has reached approximately 100° (Figure 4.3).

We therefore reimplemented the coupling explicitly to reproduce this sequential behaviour in simulation. The policy command for each coupled finger is interpreted over a total travel of approximately 180° (exactly as hardware, which sums the total travel of both J1 and J2). During the first 100° , the command is applied to J2 while J1 is held at zero. Once J2 reaches this point, the remaining command is applied to J1, up to its approximately 80° range. The reverse ordering is used during uncurling, with J1 returning before J2 begins to move.

This correction is a prerequisite for the transfer results presented later in this dissertation. To our knowledge, no prior work using this coupling mechanism has identified or corrected this mismatch, and results obtained under the default coupling therefore cannot transfer to hardware without an equivalent correction. This has a broader implication: simulation-only results (in Isaac Lab) reported on this hand under the default coupling model may not carry over to hardware as they stand, and would require the same correction before transfer could be expected to succeed

Chapter 5

Quantifying the Sim-to-Real Gap

The transfer gap $\delta(\pi)$ introduced in Chapter 3 describes the difference between the behaviour of a policy in simulation and on the physical hand. Before attempting to reduce this gap, we first examine where the difference can arise in the system considered in this work. The same command can produce different joint motion on the physical hand, the same contact can produce different tactile observations, and the physical interaction between the hand and the Baoding balls can differ from that represented in simulation.

We therefore consider the gap through three components: the *proprioception gap*, the *tactile gap*, and the *dynamics gap*. This decomposition is used to organise the analysis in this work and is not intended as a general classification of sim-to-real gaps. For each component, we first describe what the mismatch represents and why it can occur in the Shadow Dexterous Hand Lite system. We then state the hypothesis for how the corresponding mismatch can be reduced. The experiments used to test these hypotheses are presented in Chapter 6.

5.1 Proprioception Gap

The proprioception gap is the difference between the joint motion produced by the same commanded trajectory in simulation and on the physical hand. The main causes considered in this work are differences in actuator gains, tendon-driven coupling, backlash, tendon compliance, and controller behaviour between the simulated and physical systems.

The proprioception gap is measured using the whole-hand sinusoidal protocol shown in Figure 5.1 (Physical hand excitation video in Appendix A.1). For each independently

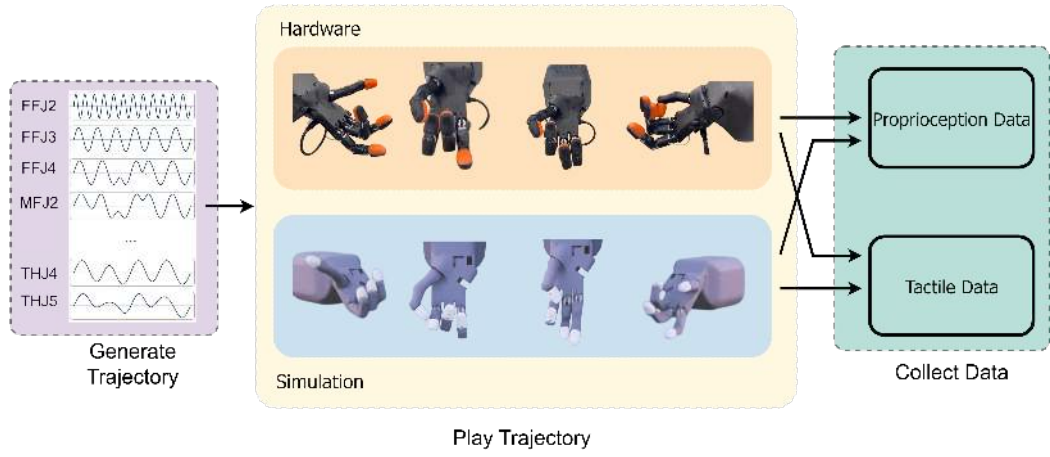


Figure 5.1: Shared data-collection protocol for the proprioception and tactile gap measurements. A whole-hand sinusoidal command is executed in simulation and on hardware, producing joint trajectories (q_{sim}, q_{hw}) and tactile signals (c_{sim}, c_{hw}). The proprioception gap is computed from the joint trajectories, while the tactile gap is computed from the corresponding tactile signals.

commandable joint, a sinusoidal trajectory with fixed frequency, number of cycles, and joint-relative amplitude is applied, while the remaining joints are held at zero. The same trajectory is executed in simulation and on hardware. The resulting traces are compared directly. Figure 5.2 quantifies this gap for RFJ3, a directly driven joint, and MFJ2, a coupled joint, as representative examples. The gap is small for most joints, while the coupled J2 joints exhibit a larger divergence between simulation and hardware.

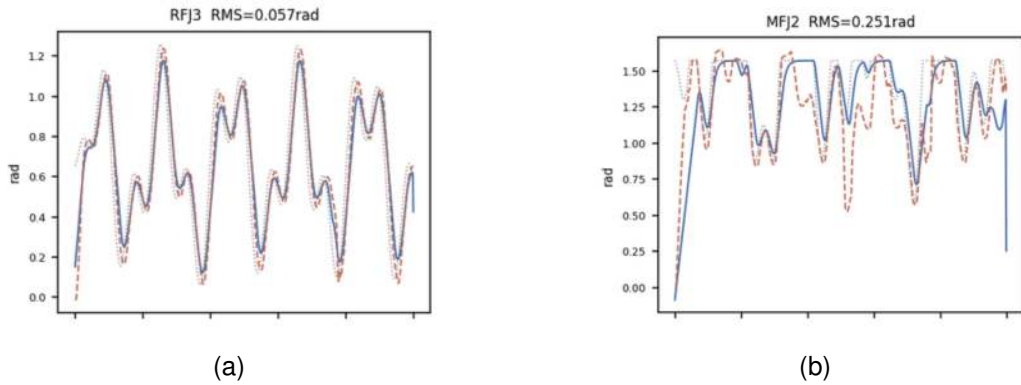


Figure 5.2: Comparison of commanded (dotted gray), simulated (blue), and hardware (orange) joint trajectories for RFJ3 and MFJ2 in the proprioception-gap measurement.

The proprioception gap is expected from the mechanical properties of the hand. The idealised PD actuator model used in simulation cannot fully capture tendon compliance,

backlash, and the sequential coupling between J1 and J2. We therefore hypothesise:

Hypothesis 1 | Proprioception Gap

The proprioception gap can be reduced by identifying the physical actuator behaviour, i.e. K_p , K_d , and F_c , more accurately in simulation.

5.2 Tactile Gap

The tactile gap is the difference between tactile observations produced in simulation and on the physical hand for corresponding contact conditions. At this stage of the experiments, the physical hand was available with TouchLab fingertips, which provided the calibrated tactile measurements used to quantify this gap. The observations and comparisons presented here are therefore based on the TouchLab configuration, but the insights gained are also applicable to the later-used BioTac and FSR configuration.

The tactile gap is measured using the same whole-hand sinusoidal protocol shown in Figure 5.1. Sinusoidal commands are applied across the fingers so that the fingertips brush against and collide with one another. The resulting tactile signals are recorded in simulation and on hardware and compared per channel. The comparison is qualitative, with attention given to channels that activate in one domain but not the other, and to channels that activate at different times.

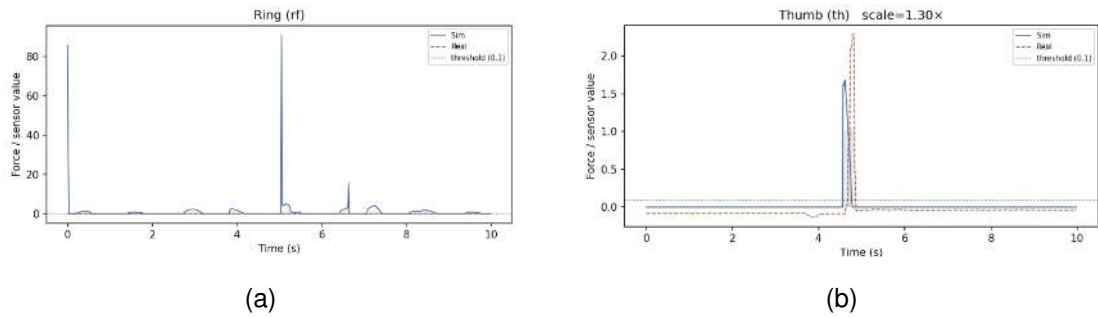


Figure 5.3: Tactile contact in simulation (blue) and on hardware (orange), measured with the TouchLab fingertip sensors. (a) Ring fingertip: the hardware channel reads zero throughout, while simulation registers spurious contacts during the same motion. (b) Thumb fingertip: both domains register contact, but the hardware response is offset in timing and roughly $1.3\times$ larger in amplitude than the corresponding simulated signal.

As shown in Figure 5.3, the simulated and hardware tactile responses differ in both the timing and occurrence of contact events. We identify two effects that are likely

to contribute to this gap. First, binary contact on hardware depends on threshold, so calibration errors can result in false positives or missed contacts even when the simulated contact state remains unchanged. Second, the physical sensor response depends on contact area, material properties, and force transmission through the fingertip, making the hardware response less consistent than the simulated contact signal. We also identified spurious contact events (Figure 5.3a) in simulation caused by allowing contact behind the fingertip. This was subsequently corrected by restricting the simulated contact region to the front of the fingertip.

Hypothesis 2 | Tactile Gap

The tactile gap can be reduced through accurate sensor calibration and by training the policy to be robust to the sensor noise and channel inconsistencies observed on hardware.

5.3 Dynamics Gap

The dynamics gap refers to uncertainty in the physical interaction parameters and contact dynamics of the task itself. This is distinct from joint-level actuator mismatch and tactile observation mismatch, as it concerns how the physical system responds during interaction. Unlike these gaps, there is no single direct measure of the dynamics gap, since it arises from multiple interacting physical parameters and contact conditions. We therefore do not quantify it as a separate scalar gap in this work.

In this work, the uncertain parameters include ball mass, ball rotational inertia, ball friction, per-link friction on the hand, and contact conditions, together with the other physical parameters included in the deployed randomisation profiles. These parameters are particularly important for contact-rich manipulation such as Baoding ball rotation, where behaviour depends continuously on ball–hand and ball–ball contact. Small differences in mass or friction can therefore change the resulting manipulation regime. These parameters are not known exactly at deployment and cannot be identified directly from motion alone.

Hypothesis 3 | Dynamics Gap

The dynamics gap can be reduced by exposing the policy to variation in relevant physical parameters, such as ball mass and friction, during simulation training.

Chapter 6

Narrowing the Sim-to-Real Gap

Chapter 5 identified three sources of the transfer gap: the proprioception gap, the tactile gap, and the dynamics gap. Having identified these sources, we now consider how each can be reduced. The aim is not to reproduce every aspect of the physical system exactly, but to identify the changes that are sufficient to make the simulated and physical systems behave similarly enough for successful policy transfer.

6.1 Narrowing the Proprioception Gap

We test Hypothesis 1 through three sequential experiments: actuator identification, coupled-joint investigation, and learned actuator dynamics. Each experiment was motivated by the outcome of the previous, with the hypothesis revised twice in response: Hypothesis 1a in Section 6.1.2 and Hypothesis 1b in Section 6.1.3. This section describes the methods used in these experiments; their outcomes are reported in the Results and Analysis chapter (Chapter 7).

6.1.1 Actuator Identification

The first experiment tests the gain component of Hypothesis 1. As no calibrated torque signal is available from the physical hand, direct torque identification is not possible. We therefore formulate the identification problem in joint position space, using the difference between the trajectories produced by the physical and simulated hand under the same command.

For each of the 13 independently commandable joints, the simulated actuator is parameterised by K_p , K_d , and F_c . The parameters are selected to minimise the difference

between the simulated and hardware joint trajectories under the same commanded motion. The identification uses four excitation signals: a chirp for the frequency response associated with K_d , a step for the transient response associated with K_p , a ramp for the near-zero-velocity regime associated with F_c , and a random trajectory for held-out validation. The amplitudes of the excitation signals are determined analytically from the rated velocity limit of each joint.

During initial data collection, joints that were not being excited were parked at zero. This produced collisions between the fingers, so the non-excited joints were subsequently parked in a fully-curved configuration. The parameter search is performed offline using a collect-then-select procedure with Latin Hypercube sampling. The offline procedure was used because the Isaac Sim rollouts stalled reproducibly; the diagnosis is given in Appendix A.4. The excitation signals and Latin Hypercube budget are specified in Appendix A.5. As the coupled J1 joints are not independently commandable and therefore inherit the identified K_p and K_d values of their corresponding J2 driver.

Gain identification reduced the mismatch only marginally on the directly driven joints and not at all on the coupled fingers (details in Section 7.1). Hypothesis 1 was therefore only partially supported: the residual coupled-finger mismatch was not attributable to the identified actuator parameters. We therefore formulated the next hypothesis.

Hypothesis 1a | Coupled-Joint Behaviour

No combination of K_p , K_d , and F_c can reproduce the coupled-finger behaviour observed on hardware. The remaining mismatch arises from how the coupling mechanism is modelled in simulation compared with how the joints actually behave on hardware, and is thus a property of the coupling mechanism rather than of the identified actuator parameters.

6.1.2 Coupled-Joint Investigation

The residual mismatch on the coupled fingers was not reduced by gain identification, so we examined the coupling representation. The sequential mechanism introduced in Section 4.2.1 corrects the original URDF/USD coupled representation, but assumes a clean handoff from J2 to J1 at 100° on hardware; this section tests whether this assumption holds on the physical hand.

6.1.2.1 Hysteresis Sweep

We first measured the coupled-joint handoff using a slow bidirectional sweep. Each coupled finger was tested independently for three cycles while the rest of the hand was held in a safe configuration. In all three fingers, J1 began moving before the nominal 100° handoff, but the amount of early motion varied between fingers (Figure 6.1).

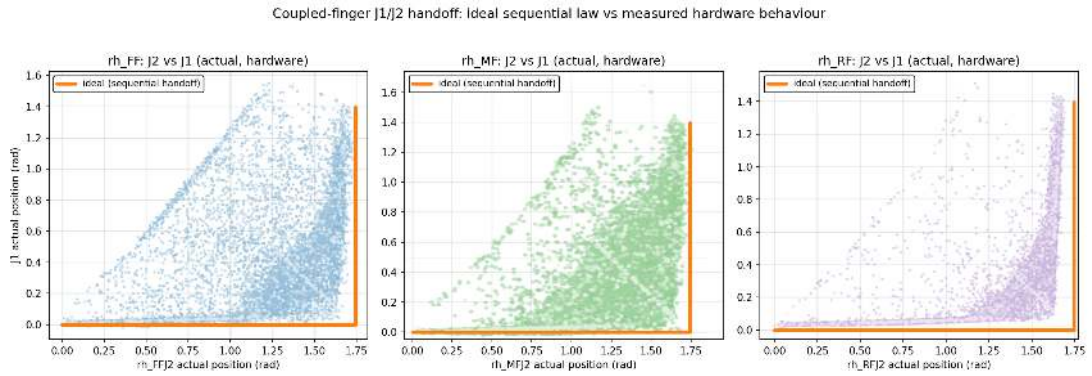


Figure 6.1: Ideal and measured coupled-joint behaviour for the FF, MF, and RF fingers. The physical hand exhibits an earlier J1 onset than the sequential handoff represented in simulation, with the deviation varying between fingers.

We then examined the shared actuator signal to understand how the actuator motion was distributed between J1 and J2. This was important because both joints are driven by the same actuator, so a torque-based model would need to determine how the shared signal is divided between them. We therefore checked whether the two joints could be treated as moving one at a time. In practice, both joints were sometimes active simultaneously, and this became more common as the sweep speed increased.

The observed behaviour, shown in Figure 6.1, is consistent with *tendon slack*. This refers to looseness in the tendon, allowing the actuator to move without immediately producing motion at the dependent joint; as the tendon becomes taut, the dependent joint can begin to move before the driver completes its nominal range. We also contacted Shadow Robotics (manufacturers of the Shadow Hand Lite; clip sent to Shadow Robotics showing the slack on hardware is linked in Appendix A.1) to understand the observed behaviour, and their feedback identified tendon slack as a likely cause.

Hypothesis 1a | Supported

J1 moves before the nominal handoff in all three fingers, consistent with tendon slack and outside what any PD parameter set can reproduce.

Since no PD parameter set can reproduce this behaviour, the coupling must be corrected by a model that adds to the PD output rather than by retuning the controller. The investigation suggests that the mismatch is a bounded, state-dependent deviation from the PD prediction, characterised by an early J1 onset that varies between fingers and increases with sweep speed. This suggests that the mismatch can potentially be captured by a learned residual on top of the fixed PD controller.

Hypothesis 1b | Learned Actuator Dynamics

A learned residual model can represent the coupled-joint behaviour that the PD model cannot. Adding a learned correction to the simulated PD output should reproduce the early-unlock behaviour observed on hardware without requiring an explicit model of the tendon mechanism.

6.1.3 Learned Actuator Dynamics

We next tested Hypothesis 1b. Three approaches were investigated. Each learns a residual torque correction on top of the simulated PD output using the same hardware dataset and data pipeline shown in Figure 6.2. The dataset contains 114 hardware episodes, resampled to 60 Hz and split by episode into training, validation, and test sets using a 0.7/0.15/0.15 split. A calibrated torque signal was not available, so the uncalibrated readout, which lies in the range $[-900, +900]$, could not be used as a direct physical torque target. For the coupled fingers, the summed actuator signal was therefore used directly without reconstructing separate joint values. The recorded topics, tactile calibration path, and alignment procedure are given in Appendix A.6. A video of the hardware data collection is linked in Appendix A.1.

6.1.3.1 Unsupervised Actuator Networks (UAN) [3]

The first approach used a reinforcement-learned residual torque model on top of the fixed PD controller. A small MLP produces 16 residual torques, which are added to the simulated PD output at each control step. The model is trained using a trajectory-matching reward against recorded hardware trajectories, so no calibrated torque target is required. The full UAN specification is given in Appendix A.7

The approach was set aside after training failed to converge (Section 7.2.1). This motivated moving to a direct position-based loss.

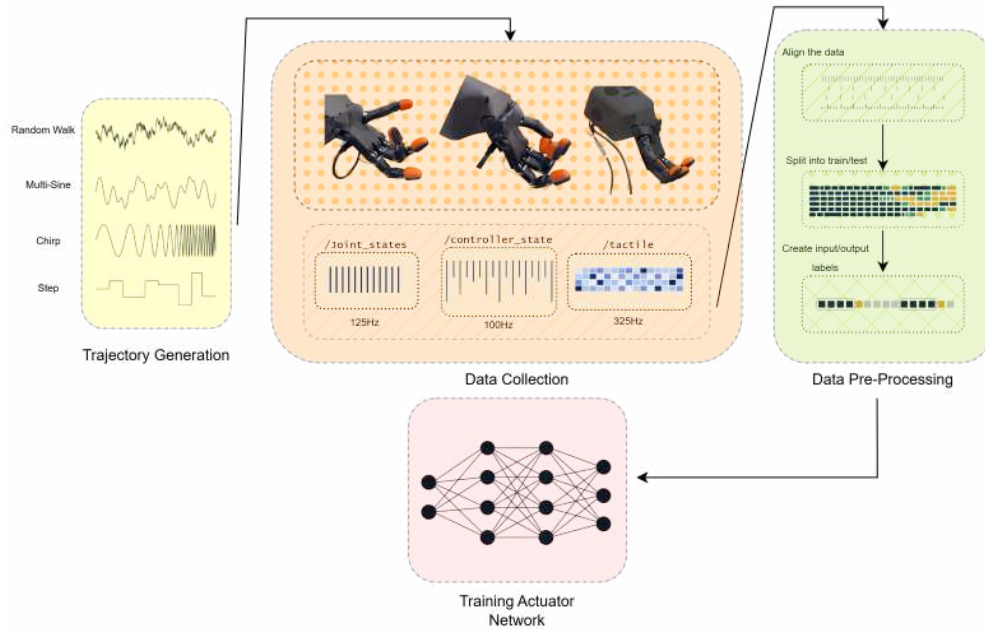


Figure 6.2: Hardware data-collection and pre-processing pipeline used for all three learned actuator-dynamics approaches. Excitation signals are executed on the physical hand under trajectory control. Joint, controller, and tactile streams are recorded, temporally aligned and resampled, and split by episode into training, validation, and test sets.

6.1.3.2 Physics-based Position Loss (Generalized Actuator Networks) [13]

The second approach moved the loss from torque space to position space. The next joint position is directly available in the hardware recorded data, so this avoids the need for a calibrated torque signal.

Isaac Lab’s PhysX tensor API provides the generalised mass matrix together with the Coriolis and gravity-compensation terms. These quantities are queried offline and used to compute the inverse-dynamics decomposition

$$\tau = M(q)\ddot{q} + C(q, \dot{q}) + G(q).$$

The predicted torque is then propagated through a differentiable semi-implicit Euler [5] step to obtain the predicted next joint position. The loss is computed against the recorded next position. The full Generalized Actuator Networks specification is given in Appendix A.8.

The approach failed on the coupled joints for the reasons described in Section 7.2.2. This motivated the next approach, which instead used the available uncalibrated actuator signal as supervision.

6.1.3.3 Learning Torque from Raw Uncalibrated Signals

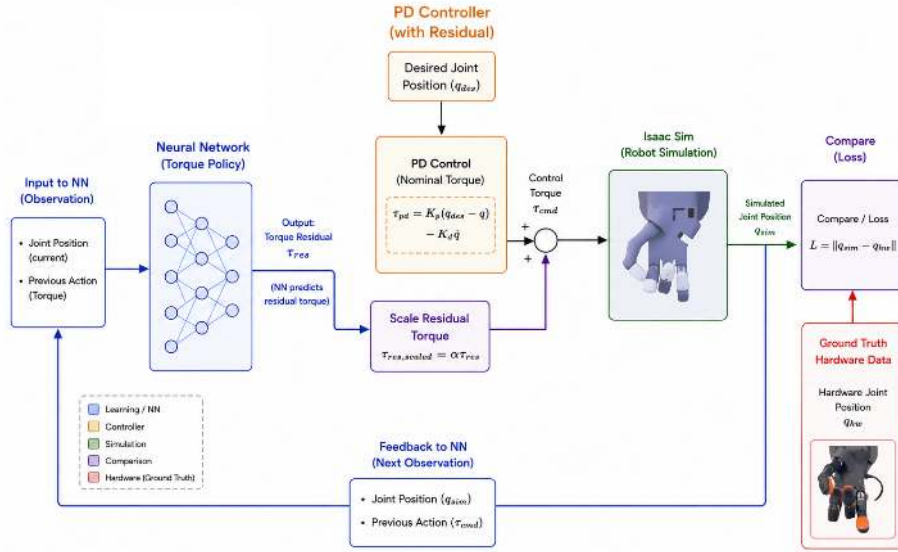


Figure 6.3: Workflow for the raw-uncalibrated-signal approach, showing the closed loop: the learned residual is added to the PD torque, drives the simulated hand, and the resulting next state feeds both the loss and the following step.

The third approach used the uncalibrated actuator signal as a training target. The full workflow of this method is shown in Figure 6.3. Although it is not expressed in physical units, the signal can still provide supervision after normalisation if a learned scale accounts for the unknown conversion to physical magnitude. For each coupled joint j , where j denotes either J1 or J2, the total predicted torque is represented as

$$\tau_{\text{total},j} = \tau_{\text{PD},j} + \alpha_j f_{\theta,j}(x_t),$$

where $\tau_{\text{total},j}$ is the predicted torque for joint j , $\tau_{\text{PD},j}$ is the torque produced by the fixed simulated PD controller, $f_{\theta,j}(x_t)$ is the learned MLP output for joint j , and α_j is a learned scale that maps the normalised MLP output to the corresponding actuator magnitude.

The MLP output is bounded to $[-1, 1]$, and the measured shared actuator signal is also normalised to this range. The scale α_j therefore accounts for the unknown conversion between the uncalibrated hardware signal and the physical torque magnitude.

Only one actuator measurement is available for the two coupled joints, so the supervision requires an attribution between them. The first strategy assumed sequential motion and assigned the shared signal to whichever joint was active. This was not

applicable during uncurling, where both joints can move simultaneously because of tendon slack.

We therefore used an activity ratio

$$r = \frac{\Delta q_{J2}}{\Delta q_{J1} + \Delta q_{J2}},$$

calculated over the next K time-steps (K was set to 10). The shared actuator measurement was then divided between the two joints using this ratio. The loss combines two mean-squared error terms for the joint shares with a direction penalty on the predicted joint torques.

The scale was estimated after MLP training by sweeping values between approximately 0 and 30 and selecting the value that produced the closest simulated trajectory. The approach was set aside because the residual intensity and its scale could not be recovered as separate unknowns; the full reasons are discussed in Section 7.2.3.

None of the three approaches produced an adopted model, so Hypothesis 1b was not supported. The coupled-joint gap was instead resolved through task scoping, restricting attention to the portion of the joint range the task actually uses so that the unmodelled region never has to be reproduced, as described in Section 6.1.4.

6.1.4 Open-loop Replay and Task Scoping

The learned actuator-dynamics approaches produced no adopted model, so the coupled-joint mismatch remained unmodelled. The coupling correction in Section 4.2.1 and the identified actuator gains from Section 6.1.1 had only been validated through joint-level comparisons. It was therefore still unknown whether these corrections were sufficient for a complete Baoding trajectory to transfer to hardware with the coupled-joint mismatch still present.

Hypothesis 1c | Open-loop Transfer

A simulation-trained trajectory will diverge from the physical hand when it enters the coupled-joint region ($100^\circ - 180^\circ$) because the remaining J1–J2 coupling mismatch is not represented in simulation.

To test this, the exact commanded trajectory from a frozen simulation policy rollout was replayed open-loop on the physical hand. Default hardware gains were used and no closed-loop correction was applied, so any divergence could not be compensated by the policy during execution.

The replay produced the intended Baoding rotation (i.e. with balls) on the hand. Inspection of the commanded trajectory showed that the J2 joints never exceeded approximately 100° . The tendon-slack region was therefore not entered, so the unmodelled coupled-joint behaviour did not affect the trajectory. The learned strategy instead pushes the balls rather than relying on deep finger flexion (see Figure 7.7), leaving the problematic region outside the motion required for the task. The mismatch therefore did not need to be modelled for this task; it was avoided by the learned behaviour without imposing a deployment constraint.

Hypothesis 1c | Not supported

The replay did not enter the tendon-slack region where the coupled-joint mismatch becomes significant. The J2 joints remained below 100° , so the unmodelled coupling did not affect the trajectory.

The same command sequence was also replayed on a second Shadow Hand Lite fitted with TouchLab fingertips. Four matched conditions were used for each hand: with and without balls, and with position-control and trajectory-control interfaces. The purpose was to assess whether the two hardware units produced comparable joint kinematics under identical commands. Unlike the BioTac-fitted hand, the task was not performed on the second unit. We therefore use the BioTac-fitted hand for the remainder of the work. The joint-level comparison is reported in Section 7.2.4.

This concludes the proprioception-gap work. Actuator identification and the learned models did not fully close the coupled-joint gap, but replaying the policy open-loop showed the task never enters the tendon-slack region, so for Baoding the coupling could be avoided rather than modelled, though a task requiring deeper flexion would still need it fixed. Fine-tuning the frozen observation encoder on hardware data was also attempted, but was not part of the transfer route reported here and did not yield a reliable improvement; it is documented in Appendix A.9.

With joint-level transfer established, the next component is the tactile channel: the binary contact signals the policy relies on to infer the balls' position, and the differences between the simulated and real sensors in producing them.

6.2 Narrowing the Tactile Gap

The tactile gap identified in Chapter 5 has two structural causes: calibration mismatch and tactile output offset. We also observed a third failure mode on the later-acquired hand, which was equipped with FSRs. Some FSR channels could become corrupted or stick during extended operation, reporting a fixed zero or one regardless of the true contact state. We therefore test Hypothesis 2 by addressing the calibration and offset in the tactile signal and by introducing targeted tactile corruption during training.

6.2.1 Addressing Calibration and Offset

The tactile offset was partially mitigated through the command-rate limiter (slew). By limiting the rate at which the hand moves (i.e., scaling the actions by a fixed factor), the ball travels across the FSR surface slowly. This makes small temporal offsets in contact registration less important, as the contact event persists for longer and can be resolved by the sensor.

The calibration mismatch was addressed at the hardware level through a warm-up procedure. Before each deployment session, a small number of ball-free Baoding motion runs were performed. The FSR noise distribution observed during these runs was used to fit a per-channel noise model. The highest observed noise value for each channel was then used as its binary contact threshold. This specific work was not part of the current thesis and is reported here for completeness and because it formed part of the deployment procedure. In simulation the threshold is effectively ideal: it is capped at a very small value close to zero, so almost every contact is registered perfectly on the FSRs. This clean signal does not match the hardware, so we deliberately reintroduce inconsistencies during training through the Tactile Corruption described in Section 6.2.2.

6.2.2 Tactile Corruption

The remaining gap was addressed during training by introducing targeted FSR corruption. At the beginning of each episode, k channels are sampled from the 12 FSR channels according to

$$k \sim U\{0, \dots, 6\},$$

where $U\{0, \dots, 6\}$ denotes the discrete uniform distribution over the integers 0 to 6 inclusive. Each selected channel is then forced to a fixed zero or one for the duration of

the episode.

The corruption is applied per episode rather than per timestep. This reflects the observed hardware behaviour, where a channel can remain stuck during an extended period of operation rather than changing independently at every timestep. The BioTac channels are left unchanged because this sticking behaviour was not observed for these sensors.

The combined effects of: the calibration procedure, the command- rate limiter, and the Tactile Corruption applied during training are evaluated in Section 7.3.2.2 and Section 7.3.2.3.

6.3 Narrowing the Dynamics Gap

The dynamics gap concerns the uncertainty in the physical parameters governing the interaction between the hand and the Baoding balls. These include ball mass, ball rotational inertia, hand–ball friction, per-link hand friction, and command-rate behaviour. Unlike actuator gains, these parameters are not directly observable from joint kinematics and therefore cannot be identified using the same motion-matching procedure. We therefore test Hypothesis 3 by introducing variation in these parameters during simulation training.

6.3.1 Closed-loop Ablation Harness

Before introducing Domain Randomisation for dynamics, we first examined which observation blocks the policy relies on. This is important because the position-error block has a large scale mismatch between simulation and hardware, approximately ± 2.4 rad in simulation compared with approximately ± 0.02 rad on hardware. An improvement following randomisation could therefore still depend on an observation that does not transfer directly between the two domains.

The four proprioceptive blocks examined were normalised joint position, normalised joint velocity, position error, and previous action. For a selected block, the corresponding values across all stacked frames were either zeroed, held at the value from the last environment reset, or replaced with independent Gaussian noise. The modified observation was then supplied to the policy and the episode was run using the same closed-loop procedure as the uncorrupted policy. A separate no-ball probe was also used to determine whether the policy had learned a motion independent of ball contact. In

this mode, both balls were placed out of reach and drop termination was disabled. This was used as the operational definition of a learned motion for the proprioception-only comparison. The results of the ablation study are reported in Section 7.3.1.2.

6.3.2 Dynamics Randomisation

We introduced randomisation at three levels. First, friction was randomised at both the hand-link and ball levels. Per-link friction was sampled independently for each episode around its nominal value, while maintaining the qualitative ordering observed on the physical hand, with the fingertips rougher than the proximal phalanges and the palm smoother (replicating the physical hardware). Ball surface friction was also sampled per episode and applied identically to both balls. The ranges were biased towards more slippery conditions to reduce overfitting to a single easy contact condition.

Second, ball mass was varied from its initial fixed value of 55 g. At each episode reset, the mass was sampled independently for each environment from 45–100 g, with both balls in an episode sharing the same sampled mass. Their rotational inertia was rescaled accordingly.

Third, a command-rate slew limiter was introduced into the simulation. At each control step, the limiter constrains the change in commanded joint position as a fraction of the rated velocity limit of each joint. This mirrors the rate limiter used during hardware deployment. The command-rate randomisation was introduced together with the ball-mass randomisation. The full domain-randomisation profiles for both policies are given in Appendix A.10.

The effect of these changes on observation reliance and hardware performance is reported in Section 7.3 for both the proprioception-only and proprioception and tactile policy.

Chapter 7

Results and Analysis

7.1 Actuator Identification

Actuator gains identified in section 6.1.1, reduced the tracking error on the directly driven joints but produced little change on the coupled fingers. Across all 13 joints, the mean absolute tracking error decreased from 0.188 rad with the placeholder gains to 0.164 rad with the identified gains. Removing an over-restrictive per-joint torque limit that was saturating one stiff joint before it reached its commanded target reduced the error further to 0.155 rad, an approximately 17% reduction relative to the placeholder configuration.

On held-out single joint data, the per-joint validation loss was between 0.01 and 0.08 across all 13 joints, indicating that the identified gains transfer at least partially beyond the single-joint conditions under which they were fitted. Figure 7.1 compares the two regimes. On the directly driven MFJ4, the simulated trajectory follows the hardware closely. On the coupled MFJ2, the hardware tracks the commanded trajectory while the simulated response continues to lag, and the identified and placeholder gains produce similar trajectories. Both simulated traces show tracking lag relative to the command, as expected when a PD controller tracks a fast policy-generated signal. These results show that actuator gains contribute to the proprioception gap on the directly driven joints, but the gain correction does not reduce the coupled-finger mismatch. This motivated the coupled-joint investigation in Section 6.1.2.

Hypothesis 1 | Partially supported

The identified gains reduced the proprioception gap on the directly driven joints but not on the coupled fingers, where the mismatch lies outside the PD model class.

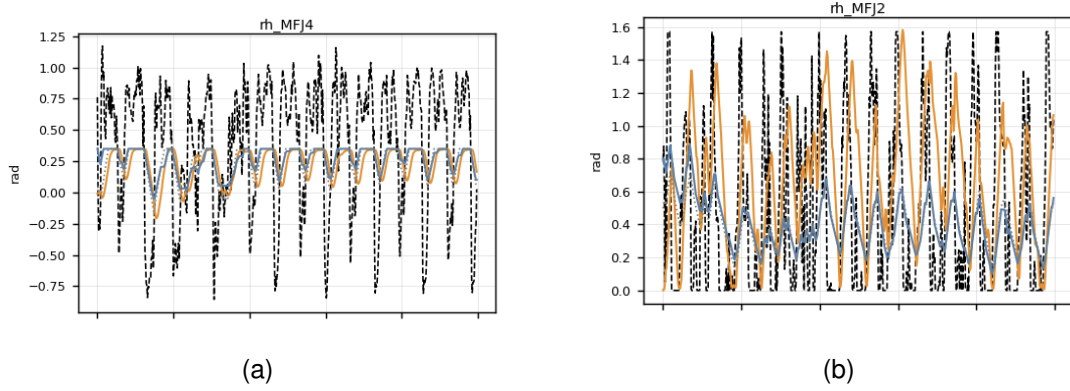


Figure 7.1: Commanded, hardware, and simulated joint trajectories under the identified actuator gains for (a) MFJ4 and (b) MFJ2. Dashed black: the policy command being replayed. Orange: hardware response. Blue: simulated response.

7.2 Learned Actuator Model

7.2.1 Unsupervised Actuator Networks

The unsupervised actuator network did not converge during training. Mean return decreased from 3.10 to -1727 by 4.79M steps and did not recover over more than 80M additional steps. Mean absolute residual torque also increased monotonically throughout training, indicating that the residual grew without improving the trajectory match.

Figure 7.2 shows the mean reward throughout training. The returns decrease sharply and do not recover. These results suggest that the trajectory-matching reward did not provide a sufficient learning signal for the residual actuator model within the available training budget. We therefore treat this as a limitation of the learning signal rather than simply insufficient training time. Only four hyperparameter trials were performed, of which one was unfinished, so it was not established whether a different reward weighting would have produced convergence. The RL-based approach was therefore set aside in favour of the direct position-based loss described in the following section and in consideration of time constraints.

7.2.2 Physics-based Position Loss

The implementation was first verified on a numerical smoke test. Given a recorded state and the true torque, it reproduced the recorded next joint position to within

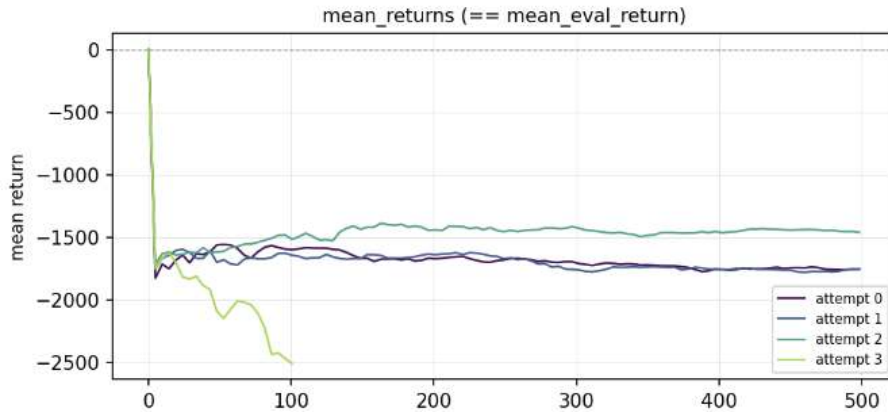


Figure 7.2: Mean reward over training for the reinforcement-learned residual actuator model.

approximately 10^{-4} rad, confirming that the inverse-dynamics computation and the integration step were correct. The approach nonetheless failed on the coupled joints, which were the primary motivation for developing the learned actuator models in the first place.

The cause is the very small identified inertia of the coupled finger links. Inertia determines how much a link accelerates for a given torque: a light link moves a great deal, while a heavier one barely responds. Converting a predicted torque into a resulting motion requires multiplying by M^{-1} , and when the inertia is small this factor becomes large. On the coupled joints, the diagonal of M^{-1} lies between approximately 15,000 and 44,000. Any error in the predicted torque is therefore multiplied by a very large number before it becomes a position, and a torque error of approximately 0.01 N m produces several degrees of position error over a single integration step.

The loss was moved into position space to avoid the torque-calibration problem, but the predicted position is still computed from a predicted torque. The sensitivity was therefore relocated instead of being removed: the network can predict the torque closely and still be scored against a substantially incorrect position. Torque-space accuracy is not a reliable proxy for position accuracy on these joints, and neither loss constrains the other. The amplification is largest on the lowest-inertia links, which are exactly the coupled joints the model was built to correct, so the position loss is least informative precisely where it is needed most. The smoke test rules out an implementation error, showing that this is a property of the problem rather than the code. A calibrated torque target does not exist on this hand, while the position loss is unusable on the coupled joints, so both torque-space and position-space supervision are closed under a fixed

model. This motivated us to return to torque space, but using the raw uncalibrated actuator signal directly as the target, with a learned scale to absorb the unknown conversion to physical units.

7.2.3 Learning Torque from Raw Uncalibrated Signals

The network was able to learn the recorded actuator signal. For the coupled FFJ1/FFJ2 pair, the predicted sum closely followed the recorded shared measurement, as shown in Figure 7.3. The predicted sum achieved an MSE of 0.0022 with 73.8% sign agreement (i.e. both coupled joints move in the same direction 73.8% of the total timesteps). Thus, within the bounded space in which the network was trained, the learning problem was tractable. The failure that follows was therefore not a failure to fit the data.

The problem is instead one of identifiability. The residual and its scale enter the dynamics only through their product, $scale_j f_{\theta,j}$. The training signal therefore constrains the product, but not the two factors individually. A given residual can be paired with a compensating scale to produce the same torque, meaning that the two quantities are not separately identifiable from the available data. Sequential estimation is therefore an incorrect methodology. Training the network at a fixed scale does not expose it to the physical effect of its own output, while sweeping the scale afterwards cannot recover the correct pairing.

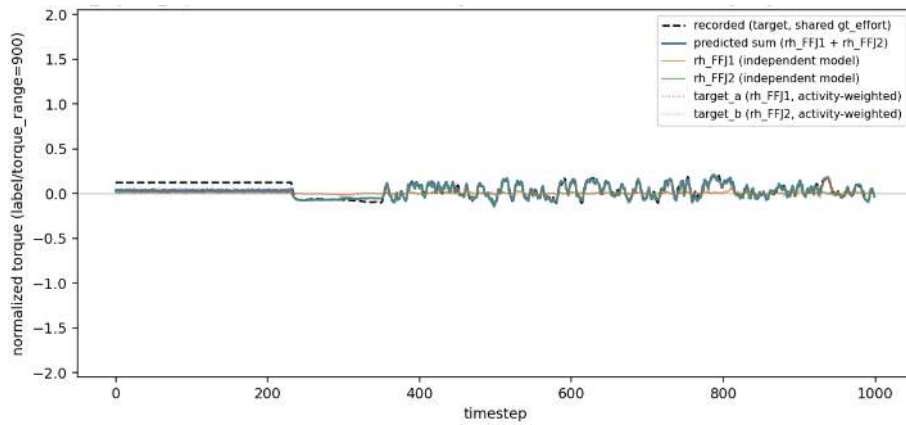


Figure 7.3: Predicted and recorded actuator signal for the coupled rh_FFJ1/rh_FFJ2 pair.

Joint estimation of both $scale_j$ and $f_{\theta,j}$ does not resolve this ambiguity with the available signals. It requires a loss that reflects the physical effect of the product. This would require either a calibrated torque target, which the physical hand does not provide, or a position-based loss. However, Section 7.2.2 showed that the latter is unusable.

Both routes are therefore closed, so the two unknowns cannot be jointly identified from the available signals.

The resulting behaviour can be seen in Figure 7.4. Once the learned residual is scaled into physical torque units and run in simulation, MFJ2 departs from the recorded trajectory and settles well outside the recorded range. The error compounds rather than remaining a fixed offset: an incorrect position at one step means that the network is evaluated at an incorrect state, so the next prediction is wrong for a new reason. MFJ1 also degrades alongside MFJ2 despite sharing the same motor and middle link. The pair therefore cannot be tuned independently, and a correct scale for one joint does not isolate the problem.

Hypothesis 1b | Not supported

None of the three learned approaches reproduced the coupled-joint behaviour that the PD model could not capture.

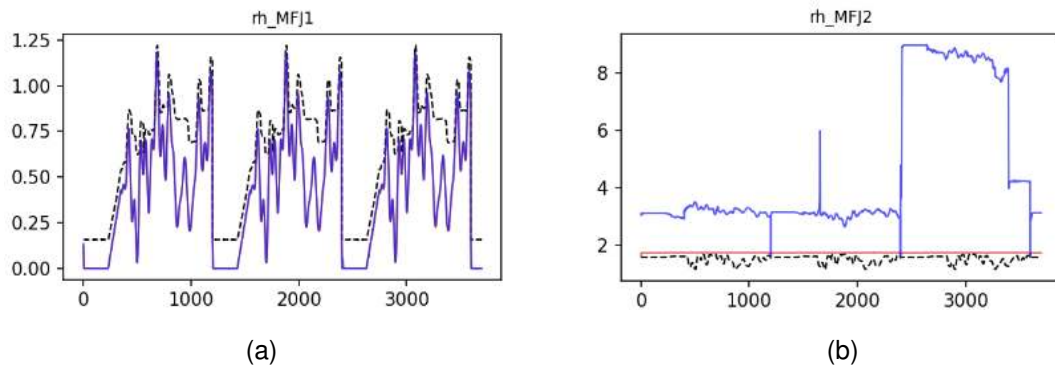


Figure 7.4: Simulated joint trajectories after the learned residual is scaled into physical torque units, for (a) MFJ1 and (b) MFJ2. Dashed black: recorded hardware trajectory. Blue: simulation under the scaled residual. Red: simulation under no residual torque.

7.2.4 Open-loop Replay

The open-loop replay produced the intended Baoding rotation on the physical hand with the balls placed. This result did not support Hypothesis 1c. Videos of the open-loop replay with the TouchLab and BioTac fingertips are linked in Appendix A.1.

As shown in Figure 7.5, the hardware followed the commanded trajectory closely across the replayed episode. On MFJ2, a coupled driver joint, the hardware tracked the command through repeated flexion–extension cycles with only a small lag. On FFJ3,

tracking was also close on the descending portion of each cycle, with the hardware slightly undershooting the commanded peaks. The close coupled-joint tracking is a direct consequence of the commanded J2 angle remaining below 100° throughout the replay, as explained in Section 6.1.4: the tendon-slack region was never entered, so the unmodelled mismatch did not affect the trajectory.

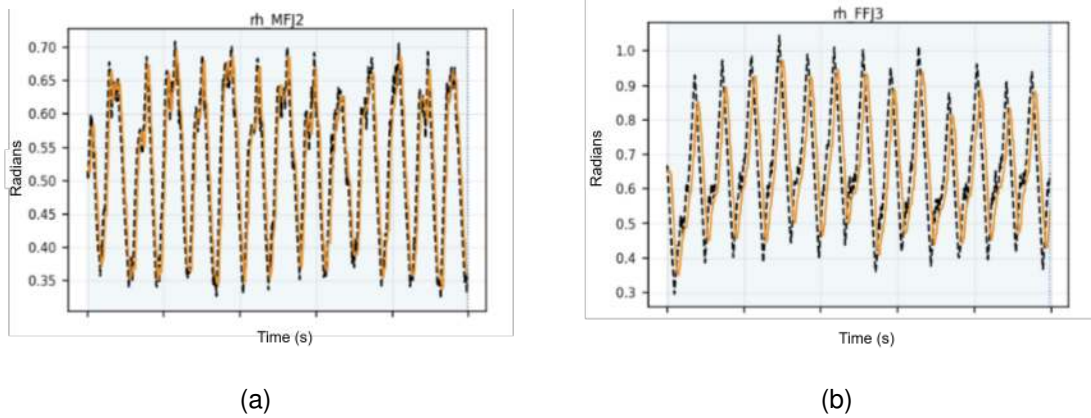


Figure 7.5: Commanded and hardware joint trajectories during open-loop replay for (a) MFJ2 and (b) FFJ3. Dashed black: replayed policy command. Orange: hardware.

The same commanded trajectory was also replayed on a second Shadow Hand Lite. As shown in Figure 7.6, the development hand (fitted with BioTac fingertips) maintained contact with both balls throughout the rotation, whereas on the second hand the balls were progressively lost and the task was not completed. The failure on the second hand began at fingertip contact rather than at the joint level. Under the same commands, the two hands showed similar joint-level kinematics. The joint recordings therefore do not identify a clear kinematic cause for the different task outcomes.

The difference most plausibly arises from fingertip contact behaviour or other mechanical properties of the two hand units that are not represented in the simulation. This result therefore indicates that matching the joint-level motion alone is not sufficient to guarantee open-loop task transfer when the task depends strongly on fingertip contact.

7.3 Hardware Transfer Progression

The hardware transfer shows a clear progression as the simulated training setup was progressively modified. Each stage exposed a different limitation of the policy and motivated the changes introduced in the subsequent stage. In this section, we evaluate both a



Figure 7.6: Open-loop replay of the same command sequence on both Shadow Hand Lite units, shown at matched points in the episode. In each pair the left image is the development hand fitted with BioTac fingertips and the right image is the second unit fitted with TouchLab fingertips.

proprioception-only policy and a policy that uses proprioceptive and tactile observations, examining how the addition of tactile information affects hardware transfer.

7.3.1 Proprioception-Only policy

7.3.1.1 Friction Randomisation

The first policy was trained with per-link and per-ball friction randomisation, but without ball-mass variation, command slew, or Tactile Corruption. In simulation, the policy achieved a mean return of approximately 250, corresponding to roughly seven rotations during a 10s episode. On hardware, however, the same policy achieved only half a rotation (See Figure 7.7). Videos of this deployment are linked in Appendix A.1. The large difference between simulation and hardware showed that friction randomisation alone was not sufficient to produce transfer.

Did Friction Randomisation help?

Friction randomisation alone was insufficient for sim-to-real transfer, with performance dropping from approximately seven rotations in simulation to only half a rotation on hardware.

7.3.1.2 Ablation and the Position-Error Signal

An ablation of the pre-randomisation proprioceptive policy explained these results. The checkpoint trained before randomisation did not meaningfully rely on the position-error

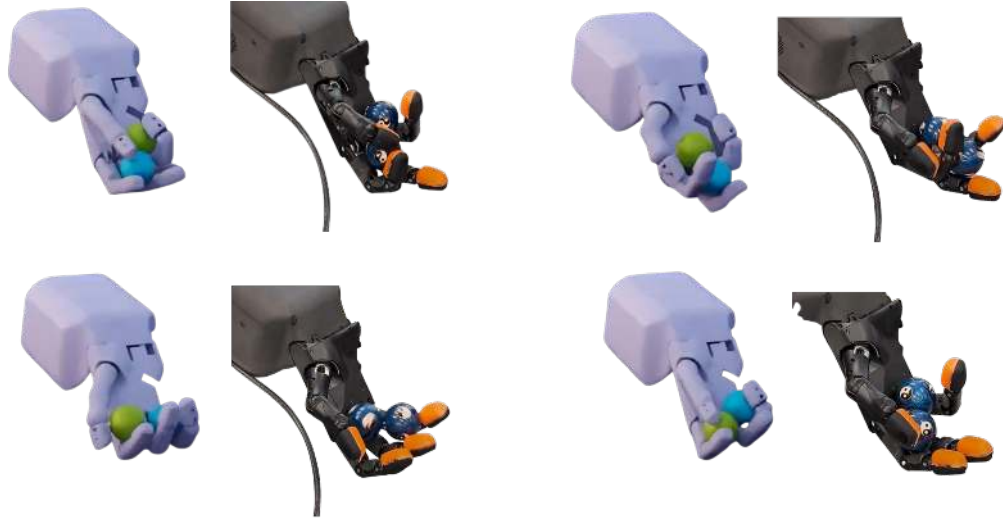


Figure 7.7: Simulation (left) and hardware (right) under the friction-only randomisation, proprioception-only policy, shown at matched points in the episode. Recorded before the coupled J2 joints were limited to avoid the tendon-slack region; the coupled J1 joints are visibly flexed in the first simulated frame (top left).

block, $e_t = q_t^{\text{cmd}} - q_t^{\text{ach}}$. However, when a ball presses against a finger, the joint may be unable to reach its commanded position. The resulting position error therefore contains information about ball position and contact force, a proprioceptive proxy for information that would otherwise be obtained through tactile sensing. A policy that ignores this block has no equivalent proprioceptive route to infer the ball state. This is consistent with the failure in Section 7.3.1.1 and the sensitivity to initial ball position observed in Stage 2. The block also has a substantial domain gap: approximately ± 2.4 rad in simulation compared with only ± 0.02 rad on hardware. We therefore changed the training objective so that the policy had a reason to use this signal, by introducing dynamics variation that could only be distinguished through the position error.

What Did the Ablation Reveal?

The position-error signal provided a proprioceptive proxy for tactile information, and its substantial simulation–hardware mismatch was identified as a key contributor to the observed transfer failure.

7.3.1.3 Ball-Mass Randomisation

Unlike the pre-randomisation checkpoint, the resulting policy did rely on the position-error block. With the ball mass varying, the contact forces also vary across episodes,

giving the position error information that helps the policy distinguish between different interaction dynamics. The policy therefore had a direct reason to use the signal that was previously being ignored. On hardware, this policy achieved continuous Baoding rotation using proprioception alone. The policy receives no tactile input, vision, or privileged simulator state, and is not trained on hardware data. Its observation consists only of joint position, joint velocity, position error, and the previous action. The result suggests that the position-error block can substitute for part of the information normally provided by tactile sensing, providing a contact-related signal that is sufficient for continuous manipulation without a tactile sensor. Video of the final proprioception-only result is linked in Appendix A.1.

What Did Mass Randomisation Reveal?

Ball-mass randomisation induced the policy to exploit the position-error signal. The policy completed 6 full 180° rotations before a ball was dropped, at a rate of approximately 1.5–2 full 180° rotations per 10 s. **To our knowledge, this is the first demonstration of continuous hardware Baoding rotation under a proprioception-only setting, with no tactile input, no vision, and no privileged simulator state.**

7.3.2 Proprioception and Tactile Policy

7.3.2.1 Friction Randomisation

Tactile sensing was then added while keeping the same friction-randomisation configuration. Hardware performance improved to approximately 2–5 rotations, but remained inconsistent, with the ball being dropped within a short period on most attempts. The behaviour was also sensitive to the initial ball position: when the balls started slightly away from the configuration expected by the policy, it was unable to adapt reliably.

7.3.2.2 After Calibration and Offset Mitigation

We next applied the hardware-side tactile corrections described in Section 6.2.1: per-channel warm-up calibration and the command-rate limiter were used during deployment. After calibration, spurious FSR activations were rarely observed in the deployment recordings, in contrast to the frequent false contacts observed before calibration. We did not measure a systematic false-positive rate, so this is reported as an observation rather than a quantified result. The same command-rate limiter was applied during

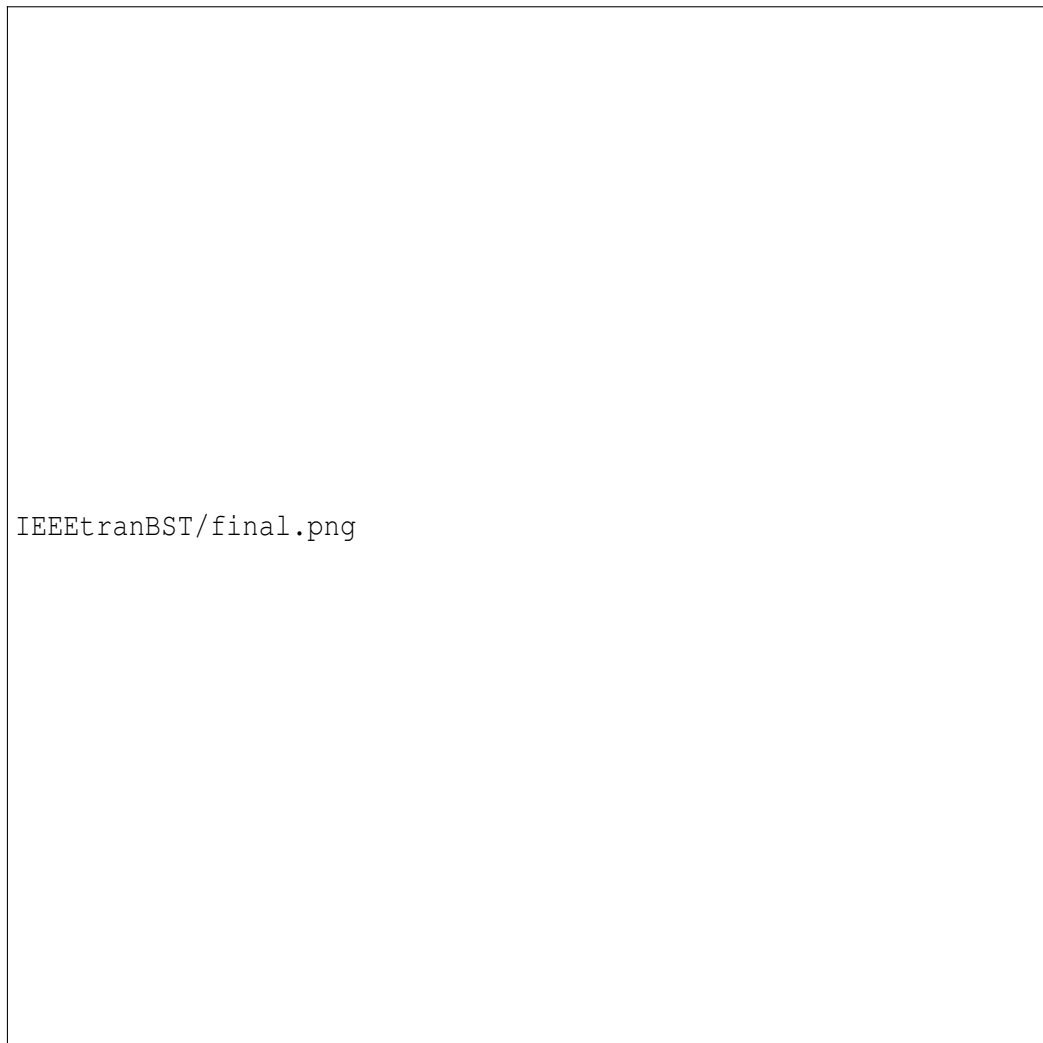


Figure 7.8: Final proprioception-and-tactile policy in simulation (left) and on hardware (right), shown at matched points in the episode.

deployment, reducing the rate at which the hand moved. This caused the balls to move across the FSR surface more gradually, so individual contact events persisted for longer. The policy therefore had more control steps in which to respond to each contact before the state changed. Fewer drops attributable to missed or late-registered contacts were observed. Thus, hypothesis 2 was partially supported at this stage.

Did Calibration and Offset Mitigation help?

Hardware-side calibration and command-rate limiting increased performance to an average of 5–12 rotations (across 8 independent runs) before a ball was dropped.

7.3.2.3 Ball Mass, Command Slew, and Tactile Corruption

Three additional randomisations were introduced in the final training configuration. Ball mass was randomised after the proprioception-only policy showed increased reliance on the position-error block under varying dynamics. Command-rate slew was moved into simulation as a training-time randomisation after rate limiting at deployment improved tactile registration in Section 7.3.2.2. This exposed the policy to the same constraint during training rather than introducing it for the first time on hardware. Per-episode Tactile Corruption was also introduced to reflect the stuck-channel behaviour observed on the physical hand.

Command slew introduced a trade-off between rotation rate and task duration. A conservative slew limit resulted in a lower rotation rate but allowed the task to be sustained for longer, while a lighter limit produced a higher rotation rate but led to an earlier drop. With both slew limits, the total number of rotations exceeded the highest result we are aware of, which is 3 rotations in 10 seconds reported by [15]. Video of the final proprioceptive-tactile policy on hardware is linked in Appendix A.1.

Did Ball Mass, Command Slew, and Tactile Corruption help?

The final proprioception-and-tactile policy completed 55 full 180° rotations (110 half rotations) in a single continuous hardware episode without dropping either ball, at approximately 4 rotations per 10 s under the conservative slew limit. A lighter slew limit raised the rate to approximately 6 rotations per 10 s at the cost of shorter sustained operation (18–20 full rotations). **Under the blind setting used in this work (no vision, no privileged simulator state, and no hardware training data) this is to our knowledge the strongest hardware Baoding result reported, in both rotation rate and sustained operation without a drop.**

Hypothesis 2 | Supported

Calibration and command-rate limiting increased the number of rotations achievable on hardware, while Tactile Corruption made the policy robust to false positives and false negatives resulting in strong hardware performance.

Hypothesis 3 | Supported

Variation in ball mass, friction, and command slew created a wide enough distribution in simulation to account for the variation observed on hardware.

Chapter 8

Conclusion

This project investigated the sim-to-real transfer of a blind Baoding manipulation policy from simulation to the Shadow Dexterous Hand Lite. Rather than treating the sim-to-real gap as a single source of uncertainty, we separated it into proprioception, tactile, and dynamics gaps and addressed each one. On the proprioception side we identified the actuator gains, corrected the coupled-joint representation to match the tendon-driven hand, and investigated three learned actuator-dynamics models. On the tactile and dynamics sides we applied hardware-side calibration, command-rate limiting, episodic Tactile Corruption, and variation in ball mass, friction, and command slew. The final blind policy achieved 55 full 180° rotations in a single continuous episode without dropping either ball, at up to approximately 6 rotations per 10 s, which to the best of our knowledge is the strongest blind Baoding result achieved on hardware.

These results should be read within the limits of the experiments. We cannot quantify exactly how much each individual change contributed, since the tactile and proprioception-only policies used different domain-randomisation profiles and several modifications were introduced together rather than in isolation. A second Shadow Hand Lite unit did not complete the task under the same commands, and we could not identify the reason. The learned actuator-dynamics problem also remains open, having been set aside under time constraints rather than resolved.

Several future directions follow from this. The contribution of each change could be isolated through controlled ablations. Generalisation across object weight and size, which was varied in simulation but not evaluated on hardware, could be tested directly on the physical hand. The learned actuator-dynamics problem could be revisited with a calibrated torque signal or a better-conditioned formulation for the coupled links.

Bibliography

- [1] Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, et al. Solving rubik’s cube with a robot hand. *arXiv preprint arXiv:1910.07113*, 2019.
- [2] Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Józefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, Jonas Schneider, Szymon Sidor, Josh Tobin, Peter Welinder, Lilian Weng, and Wojciech Zaremba. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020.
- [3] Nolan Fey, Gabriel B Margolis, Martin Peticco, and Pulkit Agrawal. Bridging the sim-to-real gap for athletic loco-manipulation. *arXiv preprint arXiv:2502.10894*, 2025.
- [4] Sicong Gao, Maurice Pagnucco, Tomasz Bednarz, and Yang Song. Nvidia isaac sim: Enabling scalable, gpu-accelerated simulation for robotics. *arXiv preprint arXiv:2606.03551*, 2026.
- [5] Krishna Murthy Jatavallabhula, Miles Macklin, Florian Golemo, Vikram Voleti, Linda Petrini, Martin Weiss, Breandan Considine, Jérôme Parent-Lévesque, Kevin Xie, Kenny Erleben, et al. gradsim: Differentiable simulation for system identification and visuomotor control. *arXiv preprint arXiv:2104.02646*, 2021.
- [6] Kang-Won Lee, Yuzhe Qin, Xiaolong Wang, and Soo-Chul Lim. Dextouch: Learning to seek and manipulate objects with tactile dexterity. *IEEE Robotics and Automation Letters*, 9(12):10772–10779, 2024.
- [7] Elle Miller, Trevor McInroe, David Abel, Oisín Mac Aodha, and Sethu Vijayakumar. Enhancing tactile-based reinforcement learning for robotic control. In *Advances in Neural Information Processing Systems*, 2025.

- [8] Elle Miller, Jayaram Reddy, Ayush Deshmukh, Trevor McInroe, David Abel, Oisín Mac Aodha, and Sethu Vijayakumar. roto 2.0: The robot tactile olympiad. *arXiv preprint arXiv:2605.21429*, 2026.
- [9] Mayank Mittal, Pascal Roth, James Tigue, et al. Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025.
- [10] Anusha Nagabandi, Kurt Konolige, Sergey Levine, and Vikash Kumar. Deep dynamics models for learning dexterous manipulation. In *Proceedings of the Conference on Robot Learning*, volume 100, pages 1101–1112. PMLR, 2020.
- [11] Jiahe Pan, Stelian Coros, Jitendra Malik, and Toru Lin. Beyond binary: Sim-to-real dexterous manipulation with physics-grounded contact representation, 2026.
- [12] Haozhi Qi, Ashish Kumar, Roberto Calandra, Yi Ma, and Jitendra Malik. In-hand object rotation via rapid motor adaptation. In *Proceedings of the 6th Conference on Robot Learning*, volume 205, pages 1722–1732. PMLR, 2023.
- [13] Jan Schneider, Mridul Mahajan, Le Chen, Simon Guist, Bernhard Schölkopf, Ingmar Posner, and Dieter Buehler. Sim-to-real transfer for muscle-actuated robots via generalized actuator networks, 2026.
- [14] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017.
- [15] Ying Yuan, Haichuan Che, Yuzhe Qin, Binghao Huang, Zhao-Heng Yin, Kang-Won Lee, Yi Wu, Soo-Chul Lim, and Xiaolong Wang. Robot synesthesia: In-hand manipulation with visuotactile sensing. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6558–6565. IEEE, 2024.
- [16] Henry Zhu, Abhishek Gupta, Aravind Rajeswaran, Sergey Levine, and Vikash Kumar. Dexterous manipulation with deep reinforcement learning: Efficient, general, and low-cost. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 3651–3657. IEEE, 2019.

Appendix A

Appendices

A.1 Supplementary videos

The behaviours described throughout this dissertation are, by their nature, easier to watch than to read about: a policy dropping a ball, a tendon going slack, or a hand tracking a replayed trajectory says more in a few seconds of video than a figure can. For that reason, video recordings for the experiments and observations discussed in the main chapters are collected and linked here, each pointing to the recording for the corresponding piece of work, hosted on the project drive.

- Data collection on hardware for actuator modelling: [click here](#)
- Tendon-slack identification, the clip sent to Shadow Robotics showing the slack developing on hardware: [click here](#)
- Data collection on hardware for the proprioception-only tactile gap measurement: [click here](#)
- First zero-shot proprioception-only deployment on hardware, default coupling (Isaac Lab), with friction randomisation: [click here](#)
- Proprioception-only deployment, new coupling, with friction randomisation only: [click here](#)
- Proprioception-only deployment, new coupling in simulation: [click here](#)
- Open-loop replay with the TouchLab fingertip sensors: [click here](#)
- Open-loop replay with the BioTac sensors: [click here](#)

- Final proprioception-only zero-shot result, six ball rotations before the drop: [click here](#)
- Final zero-shot proprioceptive-tactile policy working on hardware: [click here](#)
- ICRA submission, the first Shadow Hand Lite result: [click here](#)

A.2 Joint-limit table for the Shadow Hand Lite

The Shadow Dexterous Hand Lite has a fixed range of travel on each of its joints, and these limits define the space any policy or replayed trajectory is allowed to move through. The lower and upper travel of every joint, in both degrees and radians, is given in Table A.1.

Table A.1: Joint limits for the Shadow Dexterous Hand Lite.

Joint(s) (Refer to Fig. 4.1b)	Degrees		Radians	
	Min	Max	Min	Max
FF1, MF1, RF1	0	90	0	1.571
FF2, MF2, RF2	0	90	0	1.571
FF3, MF3, RF3	-15	90	-0.262	1.571
FF4, MF4, RF4	-20	20	-0.349	0.349
TH1	-15	90	-0.262	1.571
TH2	-30	30	-0.524	0.524
TH4	0	70	0	1.222
TH5	-60	60	-1.047	1.047

A.3 Full identified actuator gains

Each joint is driven by a PD controller, and the gains that controller uses differ between simulation and hardware. In simulation they are held fixed, and on hardware they follow from applying a per-joint stiffness and damping multiplier to those fixed values. The full set is given in Table A.2. J1 and J2 of each finger are driven by the same motor, so they necessarily share the same K_P and K_D values.

Table A.2: PD controller gains used in simulation and hardware. The simulation gains are fixed at $K_P = 1$ and $K_D = 0.1$. Hardware gains are obtained by applying the corresponding stiffness and damping multipliers.

Joint	Simulation		Hardware		
	K_P	K_D	Multiplier	K_P	K_D
FFJ1, MFJ1, RFJ1	1	0.1	0.22	0.22	1
FFJ2, MFJ2, RFJ2	1	0.1	0.30	0.30	1
FFJ3, MFJ3, RFJ3	1	0.1	0.22	0.22	1
FFJ4, MFJ4, RFJ4	1	0.1	0.40	0.40	1
THJ1	1	0.1	0.20	0.20	1
THJ2	1	0.1	0.20	0.20	1
THJ4	1	0.1	0.20	0.20	1
THJ5	1	0.1	0.20	0.20	1

A.4 Isaac Sim rollout-stall diagnosis

The identification of the PD and Coulomb-friction parameters was originally meant to run as an adaptive black-box search, where the optimiser proposes a candidate, simulates it, looks at the result, and proposes the next candidate accordingly. That plan was abandoned in favour of an offline collect-then-select procedure (generate all candidates first, simulate them, then pick the best) because the simulation rollouts stalled reproducibly partway through, even on short single-joint runs of three seconds and 180 samples.

The stall was diagnosed rather than assumed. What mattered was that every trial eventually finished, and finished with the correct output; the problem was a severe but bounded stall somewhere inside the simulation stack, not a deadlock and not a bug in our own code. That distinction is what ruled out the adaptive search: a search that has to wait on each simulation before choosing the next one cannot tolerate an unpredictable multi-minute stall on every one of the hundreds to thousands of evaluations it needs, whereas an offline batch can simply be left to run and resumed if interrupted. The hypotheses we tested, and what each one showed, are given in Table A.3.

Table A.3: Diagnosis of the Isaac Sim rollout stall.

Hypothesis	Test performed	Result
The simulation step itself is slow	Isolated diagnostic outside the identification codebase	Fast (50–90 steps/s)
Stall is gain-dependent	Default-gain rollout compared against a $3\times$ -gain rollout	Stalled at the identical sample index in both
Headless rendering overhead	Rendering disabled explicitly	No change
Network or telemetry timeout	Direct reachability check to the endpoints	Instant response
GPU clock throttling	GPU state inspected during a rollout	GPU at maximum performance state

A.5 Excitation-signal specification and Latin Hypercube configuration

To identify a joint’s parameters, that joint first has to be moved in a way that actually reveals them; a single motion does not excite stiffness, damping, and friction equally. Four different excitation families were therefore applied to each joint, each chosen to bring out one component of $\theta = (K_p, K_d, F_c)$ more than the others. The amplitudes were not tuned by hand but derived analytically from each joint’s own rated velocity limit (through $A \cdot 2\pi f$), so that every joint was driven as hard as it is allowed to move and no harder. The four families, and what each one primarily reveals, are listed in Table A.4.

Table A.4: Excitation signal families.

Signal	Description	Primarily identifies
Chirp	Frequency sweep, 0.05–2.5 Hz	K_d (damping)
Step	Instantaneous setpoint change, no slew limiting	K_p (stiffness)
Ramp	Slow quasi-static sweep across the joint range	F_c (Coulomb friction)
Random	Superposition of sinusoids at randomised frequencies, one held out per joint for validation	Combinations not covered above

With the excitation data recorded, the parameters themselves were chosen by scoring candidates against it. The candidates (θ vectors) were drawn using Latin Hypercube sampling, which spreads a fixed number of samples evenly across the three-parameter space rather than clustering them, at a deliberately modest budget of 40 per joint. The budget is small for a simple reason: each candidate costs one full simulation rollout to evaluate. Every candidate was scored against the joint’s recorded chirp, step, and ramp data, and each result was written to persistent storage the moment it was computed, so the batch could be interrupted and resumed around the stall described in Appendix A.4.

A.6 Data pipeline in full

Everything downstream of the hardware depends on knowing exactly what the hand publishes and how fast, so the hand’s ROS-level topology (the set of data streams the robot broadcasts) was confirmed live rather than taken from documentation, and every excitation episode was recorded in full. The streams that were recorded, and the rate at which each arrives, are given in Table A.5. We used the TouchLab fitted hand for Actuator Modelling.

Table A.5: Recorded topics and rates.

Stream	Content	Rate
Joint states	16-joint position, velocity, effort	~ 125 Hz
Per-actuator controller state ($\times 13$)	Set point, process value, its derivative, error, and command	~ 87 Hz
Calibrated tactile	64-taxel array (4 fingers $\times$ 16 taxels)	~ 325 Hz
Diagnostics	Controller and hardware health	—

A.6.0.0.1 Tactile calibration path. The tactile signal needs a little care, because the raw form the hand publishes is not the form the policy consumes. The raw tactile topic reports 19 uncalibrated electrode counts per finger. A calibration node takes this and publishes a 240-value message, which decomposes into 80 taxel positions of three components each, of which only the third (the normal-force component) is meaningfully non-zero when the hand is at rest. Collection was switched from the raw topic to this calibrated one once it became clear the raw signal does not match what the deployed policy actually reads at inference; using it would have meant training and deploying on different quantities. The pipeline then drops the little finger, which is not wired, and keeps 16 taxels on each of the remaining four, for 64 channels in total.

A.6.0.0.2 Alignment and windowing. The recorded streams arrive at different rates, so before they can be used together they are resampled onto a single uniform 60 Hz grid, which matches the deployed policy’s control rate, using zero-order hold (each value held until the next arrives). Any grid point more than 50 ms away from a real sample is marked invalid, and every point is tagged with a contiguous-run identifier so that the later windowing step never accidentally stacks frames from either side of a gap into one sample. The aligned columns are then windowed into policy-shaped samples and normalised with a z-score fitted only on the training split. That split is a seeded 0.7/0.15/0.15 division made by whole episode, so no window from one episode’s motion can leak across the boundary between training, validation, and test.

A.7 UAN full specification

UAN was the first of the three learned actuator models we tried, and unlike the one that followed it, it was trained by reinforcement learning rather than against a recorded label.

A PPO policy outputs 16 residual torques, which are added on top of the simulator’s own implicit PD command ($K_p = 1.0$, $K_d = 0.1$); the idea is that the network learns only the correction the idealised controller is missing. Its observation is 58 numbers per frame, stacked over the last 20 frames into a 1160-number input, so that the network sees a short history rather than a single instant. The observation layout, the reward terms, and the run trace are given in Tables A.6, A.7, and A.8.

Table A.6: UAN observation, 58-d per frame, stacked over 20 frames into a 1160-d encoder input.

Component	Dim	Description
Joint position	16	Kinematic degrees of freedom
Joint velocity	13	One per physical motor
Joint position error	16	Raw commanded minus measured position
Action	13	Per-motor setpoint the policy is chasing

Table A.7: UAN reward terms and default coefficients.

Term	Scale	Coefficient
Survival	0.0	—
L1 (mean absolute tracking error)	−1.5	—
Exp-L2, loose	4.0	100.0
Exp-L2, normal	4.0	300.0
Exp-L2, strict	5.0	1000.0
Action-rate smoothness	0.5	0.5
Torque-sign agreement	0.0 (off by default)	—

Table A.8: UAN default run trace, read directly from the training event log.

Step	Mean returns	Position RMSE	Mean abs. residual
0	3.10	—	—
4,791,204	−1727.4	151.9	6.79
9,582,408	−1638.0	144.3	7.39
14,373,612	—	142.2	9.20
76,659,264	−1618.6	140.6	16.07
86,241,672	−1601.8	139.5	16.94

The trace shows why UAN was set aside. The returns collapse from 3.10 at the first step to −1727 within 4.79 M steps and never recover, and the mean absolute residual grows steadily rather than shrinking toward a small correction, which is the opposite of what a working residual model should do. Rather than learning the small torque the idealised controller was missing, the network was drifting further from it as training went on.

A.8 Generalized Actuator Nets (Position Loss approach) full specification

Generalized Actuator Nets (GenAN) was the second attempt, and it replaces UAN’s reinforcement-learning objective with an ordinary supervised one: instead of being rewarded for good tracking, the network is simply trained to reproduce the torque actually recorded on hardware. It is an ensemble of five MLPs, each initialised independently and each with two hidden layers of 512 tanh units, all sharing a single input scaler and a single label scaler. Its inputs are delta-histories of measured and commanded position (each past value expressed as its difference from the current one) for both streams, following the source paper’s finding that this representation outperforms a raw, sparsely-strided history. The loss is a standardised-space mean squared error against the recorded effort.

Here too the coupled joints are the difficulty. The direct joints converge cleanly, but FFJ2, which shares its motor with FFJ1, settles on a torque scale an order of magnitude larger than the direct joints and never stabilises, its RMSE oscillating instead of converging (Table A.9). This is the same coupled-joint attribution problem

that appears in the shared-trunk and independent pair-model sign-agreement figures, showing up here from a third, independent direction: when two joints share one motor, deciding how much of the measured torque belongs to each is genuinely ill-posed.

Table A.9: Torque-scale sweep convergence, final iterations.

Joint	Converged scale	Converged RMSE	Notes
FFJ3, direct	≈ 0.489	≈ 0.049	Clean convergence
FFJ1, coupled	≈ 26.39	≈ 0.086	Clean convergence
FFJ2, coupled	≈ 29.19	1.8–4.4	Oscillates, never settles

FFJ2 shares its motor with FFJ1. Its converged scale oscillates and its RMSE does not settle.

A.9 Encoder fine-tuning attempt

This section describes a method that does not appear in the main text, because it was not part of the transfer route we ultimately chose. Once open-loop replay was selected as the way to take a policy onto hardware, a natural question followed: rather than leave the policy’s frozen observation encoder exactly as it was trained in simulation, could that encoder be fine-tuned on real hardware data so that its internal representation of a state matched between simulation and the real hand, and the domain gap shrank at the level of the features the policy actually consumes? The work below was an attempt to answer that. It is placed in the appendix rather than the main text because it was exploratory and, in the end, did not yield a reliable improvement; it is recorded here in full because the reasons it did not work are themselves informative.

Before any fine-tuning could be trusted, the offline reconstruction of the hardware data had to actually match what the network had seen in simulation, and three discrepancies were found and corrected first (Table A.10). Each of these is the kind of silent mismatch that does not throw an error but quietly trains the network on the wrong quantity.

Table A.10: Data-fidelity fixes applied to the offline reconstruction pipeline.

Fix	Discrepancy found	Correction
Coupled-joint normalisation range	Combined range $[0, \pi]$ used instead of the actual training range $[0, 1.745]$ rad; sampled point differed by -0.608 against -0.288	Single constant change plus a defensive clip
Hardware last-action reconstruction	Approximated as normalised commanded position against the actual raw action range $[-11.8, 7.1]$	Sequential replay of the frozen encoder and policy head over the recorded stream
True simulated velocity	Finite-difference approximation differed from the true physics-engine velocity by up to 24 rad/s	True velocity read directly where available

The last-action fix was verified rather than assumed correct: replaying the frozen network over a simulated trajectory’s own recorded observations reproduces that trajectory’s actual next action to a maximum absolute difference of 3.1×10^{-6} , which is float32 precision, against 8.5 for the naïve unshifted pairing. In other words, the corrected reconstruction is exact to the limits of the number format, and the old one was not.

Simulated and hardware episodes are not the same length (simulation runs a fixed 600 frames, hardware runs 1725–1760), so before they can be compared frame to frame they have to be aligned in time. This was done with dynamic time warping, which finds the best non-linear correspondence between two sequences of different lengths, implemented directly as a dynamic program over achieved joint position because no suitable third-party implementation was available. Aligning on achieved position rather than commanded position matters: the commanded signal is around nine times noisier frame-to-frame in simulation, and aligning on it degrades the match. Aligning on achieved position gave a mean aligned RMS of 0.0799 across ten seeds, against 0.2455 for the naïve same-index baseline that simply pairs frame i with frame i .

One further correction was needed on the loss itself. The frozen encoder’s latent dimensions turned out to have a 6.4-fold spread between the highest and lowest variance, which let a handful of high-variance dimensions dominate the gradient and effectively drown out the rest. Per-dimension whitening (rescaling each dimension to unit variance)

was applied to correct this, and a controlled re-run confirmed that the whitening does not, in itself, destabilise training.

With all of this in place, the fine-tuning still did not deliver. The domain gap was first broken down by observation block to see where it actually lived (Table A.11), and tactile came out as the single largest identified contributor, at roughly 14% against roughly 9% for position error.

Table A.11: Block-ablation of the domain gap: contribution of each observation block.

Block	Δ gap when zeroed	Sim/hw scale ratio
Position	+0.024 (helps)	1.02
Velocity	+0.001 (neutral)	2.96
Position error	−0.107	21.45
Last action	+0.047 (helps)	1.13
Tactile	−0.165	—

A twelve-configuration sweep over learning rate, rollout horizon, and discount factor was then run on a tactile-free checkpoint (proprioception only) to see whether fine-tuning could be made to converge cleanly on the simpler input. The best configuration reached a held-out one-step gap of 0.1776 at epoch 4, but every one of the twelve showed the same shape regardless of discount factor or horizon: an early peak somewhere around epochs 1–13, followed by steady degradation. When the outcome is that insensitive to the hyperparameters, the hyperparameters are probably not the cause. The most likely explanation is a data-provenance mismatch: this checkpoint was fine-tuned on a dataset that had been collected under a different policy than the one being fine-tuned, so the states in the data do not quite match the states the checkpoint itself would visit. Once the genuinely transferable signal in that mismatched data is exhausted, the encoder appears to start fitting the idiosyncrasies of the particular trajectory instead, and the gap grows again. This is what pointed us back toward open-loop replay as the more dependable route.

A.10 Configurations, checkpoints and DR profiles

The proprioception-only and proprioceptive-tactile policies were developed as two separate lines of work, with different observations, different checkpoints, and different

domain-randomisation needs, so their configurations are recorded separately here rather than collapsed into one profile. Both lines share the same physical randomisation, over ball mass, per-link hand friction, and ball friction, and both apply a command-rate slew clamp; the tactile line adds one further term, taxel corruption, that has no meaning for the proprioceptive policy. The friction and mass randomisation is common to both, so it is described once first.

A.10.0.0.1 Ball mass and ball friction. The ball mass is sampled per environment per reset over 45–100 g, around a baseline of 55 g, with the ball’s inertia rescaled to stay physically consistent with the sampled mass. Layered under this is a separate ball-friction randomisation: a single friction coefficient is sampled per environment per reset over 0.2–0.6 and applied identically to both balls, so the two share one coupled coefficient rather than being randomised independently. This shared application is deliberate, since the standard per-body material randomisation cannot express “the same sampled value across two separate objects”; the value is therefore written directly into both balls’ material buffers. Restitution is held at zero throughout, so the dynamic and static coefficients coincide and the term is effectively a single friction value per environment.

A.10.0.0.2 Per-link hand friction. The hand’s own friction is randomised per segment rather than as one global value, because a fingertip gripping a ball and a palm resting behind it do not have the same contact behaviour and should not be forced to share a range. Five per-segment terms are resampled every episode reset, each with its own static and dynamic friction range, drawn from 250 buckets and constrained so that the sampled dynamic coefficient never exceeds the static one (Table A.12). All five use zero restitution.

Table A.12: Per-link hand friction randomisation (ShadowLiteFrictionEventCfg). Five per-segment terms, resampled per reset, with sampled dynamic friction constrained to not exceed static.

Segment	Targets (body regex)	Static	Dynamic
Distal	rh_.*distal (fingertips gripping the balls)	0.8–1.5	0.7–1.3
Middle	rh_.*middle	0.7–1.2	0.6–1.0
Proximal	rh_.*proximal	0.7–1.2	0.6–1.0
Knuckle	rh_(.*knuckle thbase) (finger knuckles and thumb base)	0.5–1.0	0.5–1.0
Palm	world (palm and forearm, merged into the fixed base during URDF-to-USD conversion)	0.3–0.8	0.3–0.8

A.10.0.0.3 Command-rate slew. Both policies also apply a command-rate slew clamp, which limits how much the commanded position is allowed to change from one step to the next, mirroring the rate limiter present on the real hardware. The clamp is expressed as a fraction of each joint’s rated velocity limit: a slew value of 0.3, for example, caps each step’s commanded change at 0.3 times what the joint could travel in that interval. Unlike the earlier generic noise randomisation, this term was left on both during training and during hardware deployment, since the hardware rate limiter is always present and the policy has to be comfortable with it. The one difference between the two lines is the deployed value: the proprioception-only policy ran with a slew of 0.4 on hardware, and the proprioceptive-tactile policy with 0.5.

A.10.0.0.4 Proprioception-only. The proprioception-only policy sees only joint state, no tactile input, and was the line taken furthest on hardware, ending in the final zero-shot result of six ball rotations before the drop. Its randomisation is exactly the shared physical set above, ball mass, ball friction, per-link hand friction, and the command-rate slew at 0.4, with no tactile term (Table A.13). This is also the line through which the coupling mechanism was revised, from the old coupling used in the first hardware deployment to the new coupling used in the friction- and mass-randomisation runs.

Table A.13: Domain-randomisation profile, proprioception-only policy.

DR source	Range / mechanism	Deployed
Ball mass	45–100 g per environment per reset (baseline 55 g), inertia rescaled to match	On
Ball friction	Single coefficient 0.2–0.6 per reset, shared across both balls, restitution 0	On
Per-link hand friction	Five per-segment static/dynamic ranges, resampled per reset (Table A.12)	On
Command-rate slew	Per-step commanded-change clamp as a fraction of rated velocity (config default off)	On, 0.4

A.10.0.0.5 Proprioceptive-tactile. The proprioceptive-tactile policy adds 12 FSR and 4 BioTac fingertips, and with that extra sensing comes an extra source of sim-to-real mismatch that has to be randomised over: the tactile signal itself. On top of the shared physical set, it therefore adds taxel corruption, which forces a random subset of physical channels on or off each episode so the policy cannot depend on any single taxel being reliable, and it runs with a command-rate slew of 0.5 on hardware (Table A.14). The final zero-shot proprioceptive-tactile policy that worked on hardware came from this line.

Table A.14: Domain-randomisation profile, proprioceptive-tactile policy.

DR source	Range / mechanism	Deployed
Ball mass	45–100 g per environment per reset (baseline 55 g), inertia rescaled to match	On
Ball friction	Single coefficient 0.2–0.6 per reset, shared across both balls, restitution 0	On
Per-link hand friction	Five per-segment static/dynamic ranges, resampled per reset (Table A.12)	On
Command-rate slew	Per-step commanded-change clamp as a fraction of rated velocity (config default off)	On, 0.5
Tactile taxel corruption	0–6 of the 12 physical channels forced to 0 or 1 per episode; BioTac channels untouched (config default off)	On